Leibniz Universität Hannover

L3S Research Center

Knowledge-Based Systems Group

L3S-Offshore-2 Frontend

Web-based Petri Net Visualization and Simulation Tool

Technical Documentation

Version 1.0

Author

Martin Krauser
martin.krauser@stud.uni-hannover.de

Supervisor

Shengrui Peng

Project Period: March 2025 – January 2026

Hannover, November 2025

Document Information

Project: L3S-Offshore-2 Frontend
Version: 1.0
Date: November 2025
Author: Martin Krauser
Institution: L3S Research Center

Version History

Version	Date	Author	Changes
1.0	November 2025	Martin Krauser	Initial version

License

Abstract

This document provides comprehensive technical documentation for the L3S-Offshore-2 Frontend, a web-based application for Petri net visualization, manipulation, and simulation. The frontend was developed as part of the L3S-Offshore-2 research project at the L3S Research Center, Leibniz Universität Hannover.

The application is built using Angular and provides the following core capabilities:

- **Visualization:** Interactive rendering of Petri nets from PNML files with automatic layout algorithms (Sugiyama)
- **Manipulation:** Drag-and-drop editing of places, transitions, and arcs with real-time token modification
- **Simulation:** Step-by-step and automated playback of firing sequences with transition highlighting
- **Analysis:** Display of place invariants and transition firing statistics from multi-run simulations
- **Offshore Scheduling:** Dedicated parameter interface for offshore wind farm installation scheduling models

The documentation covers the system architecture, key implementation details, deployment procedures, and provides guidance for future development and maintenance.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Context

The L3S-Offshore-2 project is a research initiative at the L3S Research Center focusing on the application of Petri nets for modeling and simulating offshore wind farm installation logistics. Efficient scheduling of vessel operations, component installations, and resource management is critical for minimizing costs and maximizing the utilization of weather windows.

This documentation describes the development of a web-based frontend application that enables researchers and practitioners to:

- Visualize complex Petri net models in an interactive browser environment
- Modify and extend existing models through a graphical user interface
- Execute and analyze simulations to evaluate scheduling strategies
- Configure domain-specific parameters for offshore logistics scenarios

The frontend integrates with a Python-based backend (Flask) that provides simulation capabilities using the `gspn4py` library for Generalized Stochastic Petri Nets.

1.2 Problem Statement

Prior to this project, the research group relied on disconnected tools for Petri net modeling:

1. **Visualization:** Desktop applications (e.g., WoPeD) that could not be easily shared
2. **Simulation:** Python scripts executed via command line

3. **Parameter Configuration:** Manual editing of configuration files

This fragmentation created barriers for:

- Collaboration between researchers
- Rapid prototyping and iteration on models
- Demonstration of results to external stakeholders

1.3 Solution Approach

The solution is a single-page web application (SPA) that consolidates all workflows into an integrated environment. The key design decisions include:

- **Angular Framework:** Chosen for its component-based architecture and TypeScript support, enabling maintainable and type-safe code
- **Fork of Existing Project:** Instead of building from scratch, the project forked the open-source "PNML-Frontend" by i-love-petrinets, which provided a solid foundation for PNML parsing and basic visualization
- **Tab-Based Navigation:** Clear separation of concerns (Build, Code, Simulation, Analyze, Save, Offshore) for different user workflows
- **Containerized Deployment:** Docker-based deployment for easy installation on research servers

1.4 Contributions

The main contributions of this project are:

1. **Extended Visualization:**
   - Viewport-centered zooming and panning with ResizeObserver integration
   - Automatic layout via Sugiyama algorithm for PNML files without coordinates
   - Tab-aware UI state management
2. **Simulation Capabilities:**
   - Animated playback of firing sequences from backend simulation
   - Manual "Token Game" mode for step-by-step execution
   - Multi-run simulation with transition firing frequency statistics

3. **Offshore-Specific Integration:**
   - Dedicated parameter input panel for offshore scheduling models
   - Integration with backend scheduling endpoints
4. **Production-Ready Deployment:**
   - Docker containerization with nginx reverse proxy
   - Deployment on L3S research infrastructure

1.5 Document Structure

This documentation is organized as follows:

Chapter 2: Provides background on Petri nets, the PNML format, and relevant web technologies (Angular, TypeScript).

Chapter 3: Introduces the UI Service for managing application state such as the active tab and selected tools.

Chapter 4: Describes the core data model: Place, Transition, Arc, and their TypeScript implementations.

Chapter 5: Explains the central data store that manages all Petri net elements and notifies subscribers of changes.

Chapter 6: Describes the overall system architecture, component hierarchy, and service dependencies.

Chapter 7: Details key implementation aspects including the zoom system, simulation logic, and token reset.

Chapter 8: Covers PNML and JSON file import/export functionality.

Chapter 9: Explains the Spring Embedder and Sugiyama layout algorithms.

Chapter 10: Describes backend API integration for simulation and planning.

Chapter 11: Covers the Docker-based deployment process and production configuration.

Chapter 12: Discusses related tools and how this project compares.

Chapter 13: Summarizes achievements and outlines future work.

Appendix A: Contains supplementary material such as API documentation and configuration files.

Chapter 2

Fundamentals

This chapter provides the theoretical and technical background necessary to understand the implementation details described in later chapters. It covers the fundamentals of Petri nets, the PNML exchange format, and the web technologies used in the frontend development.

2.1 Petri Nets

Petri nets are a mathematical modeling language for the description of distributed systems. Originally introduced by Carl Adam Petri in his 1962 dissertation [5], they provide a graphical and formal representation of concurrent processes. For a comprehensive survey of Petri net properties and applications, see Murata [4].

2.1.1 Basic Definitions

A Petri net is a directed bipartite graph consisting of:

- **Places** (P): Represented as circles, places hold tokens and model conditions or states
- **Transitions** (T): Represented as rectangles or bars, transitions model events or actions
- **Arcs** (F): Directed edges connecting places to transitions (input arcs) or transitions to places (output arcs)
- **Marking** (M): The distribution of tokens across places, representing the system state

Formally, a Petri net is defined as a tuple $N = (P, T, F, W, M_0)$ where:

- P is a finite set of places

- T is a finite set of transitions, $P \cap T = \emptyset$
- $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation
- $W : F \to \mathbb{N}^+$ is the arc weight function
- $M_0 : P \to \mathbb{N}$ is the initial marking

2.1.2 Firing Rules

A transition $t \in T$ is **enabled** at marking M if and only if:

$$\forall p \in {\bullet t} : M(p) \geq W(p, t)$$

where ${\bullet t}$ denotes the preset of t (input places). When an enabled transition fires, the marking changes according to:

$$M'(p) = M(p) - W(p, t) + W(t, p)$$

2.1.3 Extensions: Stochastic Petri Nets

For modeling real-world systems with timing, Generalized Stochastic Petri Nets (GSPNs) [3] extend basic Petri nets with:

- **Timed transitions**: Fire after a random delay drawn from an exponential distribution
- **Immediate transitions**: Fire instantaneously when enabled (priority over timed transitions)

The L3S-Offshore-2 project uses GSPNs to model the stochastic nature of offshore installation operations, including weather-dependent delays.

2.2 PNML: Petri Net Markup Language

PNML (Petri Net Markup Language) is an XML-based interchange format for Petri nets, standardized as ISO/IEC 15909-2 [8]. It provides a common syntax for storing and exchanging Petri net models across different tools.

2.2.1 Document Structure

A PNML document has the following hierarchical structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<pnml>
  <net id="net1" type="http://www.pnml.org/version-2009/grammar
    /ptnet">
    <page id="page1">
      <place id="p1">
```

```
        <name><text>Place 1</text></name>
        <initialMarking><text>1</text></initialMarking>
        <graphics><position x="100" y="100"/></graphics>
      </place>
      <transition id="t1">
        <name><text>Transition 1</text></name>
        <graphics><position x="200" y="100"/></graphics>
      </transition>
      <arc id="a1" source="p1" target="t1">
        <inscription><text>1</text></inscription>
      </arc>
    </page>
  </net>
</pnml>
```

Listing 2.1: Basic PNML structure

2.2.2 Graphical Information

PNML files may optionally contain graphical information (positions, dimensions) in `<graphics>` elements. When this information is missing or invalid, the frontend applies the Sugiyama algorithm to generate a readable layout automatically.

2.3 Web Technologies

The frontend is built using modern web technologies that enable rich, interactive single-page applications.

2.3.1 Angular Framework

Angular [1] is a TypeScript-based web application framework developed by Google. Key concepts used in this project include:

- **Components**: Reusable UI building blocks with encapsulated logic, templates, and styles
- **Services**: Singleton classes for shared state and business logic, injected via dependency injection
- **Observables (RxJS)** [6]: Reactive programming patterns for handling asynchronous events and data streams
- **Modules**: Organizational units that group related components, services, and other code

2.3.2 SVG for Petri Net Rendering

Scalable Vector Graphics (SVG) is used for rendering Petri nets because:

- Resolution-independent rendering at any zoom level
- DOM integration allows event handling on individual elements
- CSS styling and animations can be applied
- Native browser support without plugins

2.3.3 Angular Material

Angular Material [2] provides pre-built UI components following Google's Material Design guidelines. The project uses Material components for:

- Tab navigation (`mat-tab-group`)
- Buttons and form controls
- Dialogs for warnings and information
- Icons and tooltips

2.4 Related Concepts

Chapter 3

UI Service: User Interface State Management

This chapter introduces the `UiService`, the central component responsible for managing the application's user interface state. Understanding this service is fundamental, as it dictates what the user sees and how the application responds to their interactions.

3.1 Problem Statement

Consider a common user action: **switching tabs**. The application has different tabs such as "Build", "Simulation", and "Analyze". When the user clicks on the "Simulation" tab:

1. The clicked button must inform the rest of the application: "We are now in Simulation mode."
2. The main drawing area must update to show simulation-specific elements (e.g., animated token flow) instead of editing tools.
3. Other UI components (sidebars, status messages) must reflect the new "Simulation" context.

Without a centralized state management approach, every component would need to independently track the current tab, leading to:

- **Inconsistency**: Components might display conflicting states
- **Tight coupling**: Components must directly communicate with each other
- **Maintenance burden**: Changes require updates across multiple files

The `UiService` solves this by being the **single source of truth** for what the user interface is currently displaying or doing.

3.2 Conceptual Overview

Think of the `UiService` as the application's **control panel** or **dashboard**. It maintains critical information about the UI state:

- **Active tab**: Which workflow is currently visible (Build, Simulation, Analyze, etc.)
- **Selected tool**: Which editing tool is active (Add Place, Move, Delete, etc.)
- **Simulation speed**: The playback rate for animation
- **Editor format**: Whether the Code tab shows JSON or PNML

Other components *subscribe* to the `UiService`—similar to subscribing to a notification channel. When the service announces a change (e.g., "The active tab is now Simulation!"), all subscribed components automatically react, ensuring the application remains responsive and consistent.

3.3 State Definitions

The application defines its UI states using TypeScript enums, providing clear, human-readable names instead of magic numbers.

```
/**
 * Represents the currently active tab in the application.
 * Each tab corresponds to a different workflow/mode.
 */
export enum TabState {
    Build,        // Graphical editing of Petri net elements
    Simulation,   // Playback and animation of firing sequences
    Save,         // Export options (PNG, SVG, JSON, PNML)
    Code,         // Text-based editing of PNML/JSON
    Analyze,      // Place invariants and analysis results
    Offshore,     // Domain-specific parameter configuration
}

/**
 * Represents the currently selected editing tool in Build mode.
 */
export enum ButtonState {
    Blitz,        // Quick-add mode
    Place,        // Add a new place (circle)
    Transition,   // Add a new transition (rectangle)
    Move,         // Drag elements to reposition
    Arc,          // Connect places and transitions
    Delete,       // Remove elements
    // ... additional tools ...
```

```
}

/**
 * Format for the Code tab editor.
 */
export enum CodeEditorFormat {
    JSON,         // Custom JSON format
    PNML,         // Standard PNML XML format
}
```

Listing 3.1: UI state enums (src/app/tr-enums/ui-state.ts)

Using enums like `TabState.Simulation` is far more readable and maintainable than remembering that "tab state 1" means simulation mode.

3.4 The UiService Implementation

The `UiService` class holds the actual state variables and provides reactive streams (Observables) that notify listeners of changes.

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';
import { ButtonState, CodeEditorFormat, TabState } from '../tr-
    enums/ui-state';

@Injectable({
    providedIn: 'root', // Singleton: one instance for the
    entire application
})
export class UiService {
    //
    ============================================================================

    // TAB STATE MANAGEMENT
    //
    ============================================================================


    /** Private variable holding the current tab */
    private _tab: TabState = TabState.Build;

    /**
     * BehaviorSubject: a special Observable that remembers its
     last value.
     * New subscribers immediately receive the current state.
     */
    private _tab$ = new BehaviorSubject<TabState>(this._tab);

    /** Public Observable for components to subscribe to */
    public tab$: Observable<TabState> = this._tab$.asObservable
    ();

    /** Getter: read the current tab synchronously */
```

```
26      public get tab(): TabState {
27          return this._tab;
28      }
29
30      /**
31       * Setter: update the current tab.
32       * Automatically notifies all subscribers via the
    BehaviorSubject.
33       */
34      public set tab(value: TabState) {
35          if (this._tab !== value) {
36              console.log(`[UiService] Tab changed: ${TabState[
    this._tab]} -> ${TabState[value]}`);
37              this._tab = value;
38              this._tab$.next(value); // Broadcast to all
    subscribers
39          }
40      }
41
42      //
    ============================================================================

43      // SIMULATION SPEED MANAGEMENT
44      //
    ============================================================================


45
46      private _simulationSpeed$ = new BehaviorSubject<number>(1);
     // Default: 1x speed
47      public simulationSpeed$: Observable<number> = this.
    _simulationSpeed$.asObservable();
48
49      /** Update the simulation playback speed multiplier */
50      public setSimulationSpeed(speed: number): void {
51          this._simulationSpeed$.next(speed);
52      }
53
54      /** Get the current speed multiplier synchronously */
55      public getAnimationSpeedMultiplier(): number {
56          return this._simulationSpeed$.getValue();
57      }
58
59      // ... additional state management for tools, editor format
    , etc. ...
60  }
```

Listing 3.2: UiService core implementation (src/app/tr-services/ui.service.ts)

3.4.1 Key Concepts Explained

@Injectable({providedIn: 'root'}) Makes `UiService` a **singleton**—only one instance exists throughout the application. This ensures all components share the same state.

BehaviorSubject<T> A special RxJS type that:

- Always holds a current value
- Immediately emits the current value to new subscribers
- Broadcasts new values to all active subscribers when `.next()` is called

Getter/Setter pattern The `get tab()` and `set tab()` methods provide a clean API. The setter automatically calls `_tab$.next(value)` to notify subscribers.

3.5 Usage Patterns

3.5.1 Changing the UI State

When a user clicks a tab button, the `ButtonBarComponent` updates the `UiService`:

```
import { Component } from '@angular/core';
import { UiService } from 'src/app/tr-services/ui.service';
import { TabState } from 'src/app/tr-enums/ui-state';

@Component({ /* ... */ })
export class ButtonBarComponent {
    constructor(protected uiService: UiService) {}

    /**
     * Called when user clicks a tab button.
     * Updates the UiService, which broadcasts to all
    subscribers.
     */
    tabClicked(tabName: string): void {
        switch (tabName.toLowerCase()) {
            case 'build':
                this.uiService.tab = TabState.Build;
                break;
            case 'simulation':
                this.uiService.tab = TabState.Simulation;
                break;
            case 'analyze':
                this.uiService.tab = TabState.Analyze;
                break;
            // ... other tabs ...
        }
```

```
    }
}
```

Listing 3.3: Setting the tab state (ButtonBarComponent)

3.5.2 Listening to UI State Changes

Components that need to react to state changes *subscribe* to the observable:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subscription } from 'rxjs';
import { UiService } from './tr-services/ui.service';
import { TabState } from './tr-enums/ui-state';

@Component({ /* ... */ })
export class AppComponent implements OnInit, OnDestroy {
    private tabSubscription: Subscription;

    constructor(protected uiService: UiService) {}

    ngOnInit(): void {
        // Subscribe to tab changes
        this.tabSubscription = this.uiService.tab$.subscribe(
    currentTab => {
            console.log('Current tab:', TabState[currentTab]);

            // React to the change
            if (currentTab === TabState.Simulation) {
                this.showSimulationControls();
            } else {
                this.hideSimulationControls();
            }
        });
    }

    ngOnDestroy(): void {
        // Always unsubscribe to prevent memory leaks
        this.tabSubscription?.unsubscribe();
    }

    // ...
}
```

Listing 3.4: Subscribing to tab changes (AppComponent)

The `$` suffix on `tab$` is a common Angular/RxJS convention indicating that the property is an Observable (a "stream" of values over time).

3.6 Data Flow Visualization

The following sequence illustrates what happens when a user switches tabs:

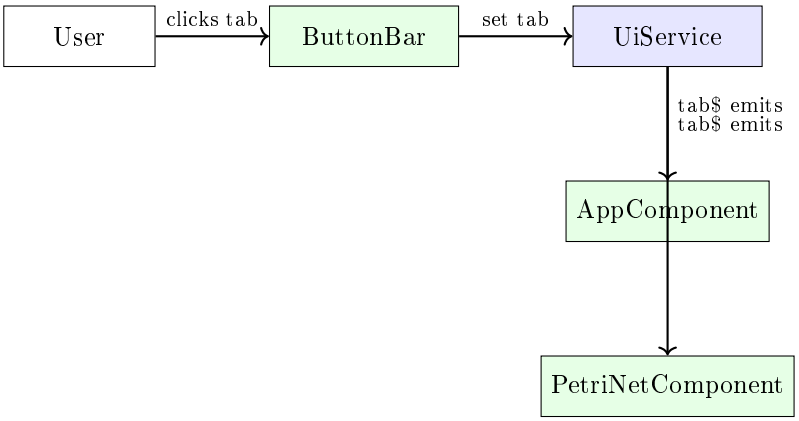

Figure 3.1: Tab switching data flow through UiService

1. **User Action**: User clicks the "Simulation" tab button
2. **State Update**: `ButtonBarComponent` sets `uiService.tab = TabState.Simulation`
3. **Internal Update**: `UiService` updates `_tab` and calls `_tab$.next(value)`
4. **Broadcast**: All subscribers to `tab$` receive the new value
5. **Component Reactions**:
   - `AppComponent` shows the simulation timeline
   - `PetriNetComponent` enables transition highlighting and hides build tools

3.7 Practical Example: Tab-Aware Behavior

The `PetriNetComponent` demonstrates how components use `tab$` to conditionally enable features:

```
import { Component, OnInit } from '@angular/core';
import { UiService } from 'src/app/tr-services/ui.service';
import { TabState } from 'src/app/tr-enums/ui-state';

@Component({ /* ... */ })
export class PetriNetComponent implements OnInit {
    private previousTab: TabState;

    constructor(private uiService: UiService) {}

    ngOnInit(): void {
        this.uiService.tab$.subscribe(currentTab => {
```

```
13              // Leaving Simulation tab: clean up highlights and
    reset tokens
14              if (this.previousTab === TabState.Simulation &&
    currentTab !== TabState.Simulation) {
15                  this.clearAllHighlights();
16                  this.resetTokensToBaseline();
17              }
18
19              // Entering Simulation tab: save baseline marking
20              if (currentTab === TabState.Simulation && this.
    previousTab !== TabState.Simulation) {
21                  this.saveBaselineMarking();
22              }
23
24              // Entering Build tab: enable editing tools
25              if (currentTab === TabState.Build) {
26                  this.enableDragAndDrop();
27                  this.showEditingHandles();
28              } else {
29                  this.disableDragAndDrop();
30                  this.hideEditingHandles();
31              }
32
33              this.previousTab = currentTab;
34          });
35      }
36
37      // ... helper methods ...
38  }
```

Listing 3.5: Tab-aware component behavior

This pattern ensures that:

- Simulation highlights are cleared when leaving the Simulation tab
- Token counts are reset to their initial values
- Editing tools are only active in Build mode

3.8 Summary

The `UiService` is the application's **control panel**, centralizing all UI state management:

- Uses **TypeScript enums** for type-safe state definitions
- Employs **BehaviorSubject** for reactive state broadcasting
- Provides **getter/setter** methods for clean state access
- Enables **decoupled components** that react to state changes without direct communication

Understanding the `UiService` is essential because it controls what the user experiences. In the next chapter, we will examine the **Petri Net Elements**—the actual data objects (places, transitions, arcs) that the UI displays and manipulates.

Chapter 4

Petri Net Elements: Core Data Model

In the previous chapter, we explored the `UiService`, which manages *what* the user is doing. This chapter focuses on *what* the user is working with: the **Petri Net Elements**—the fundamental data objects that define the structure and behavior of any Petri net in our application.

4.1 Problem Statement

To model a process like "sending a letter," we need to represent:

- **States**: The letter is unstamped, stamped, or mailed
- **Actions**: Stamp the letter, mail the letter
- **Flow**: How actions connect states

The **Petri Net Elements** are the fundamental building blocks that answer these questions. They are the "nouns" (states/conditions), "verbs" (actions/events), and "arrows" (connections) of our process models.

4.2 Core Building Blocks

Petri nets consist of three element types:

4.3 Places: State Containers

Places are represented as circles. They hold **tokens** (visualized as small dots inside the circle) and represent conditions, resources, or states.

- A place with tokens indicates that the condition is **active** or the resource is **available**

Element	Visual	Represents	Analogy
Place	Circle (◯)	States, conditions, resources	Container, location
Transition	Rectangle (□)	Events, actions, activities	Action, step, verb
Arc	Arrow (→)	Causal links, token flow	Pathway, connection

Table 4.1: The three core Petri net element types

- Example: A place named "Coffee Beans" with 5 tokens means 5 portions are ready

```
import { Point } from './point';
import { Node } from 'src/app/tr-interfaces/petri-net/node';

/**
 * Represents a Place in a Petri net.
 * Places hold tokens and model conditions/states.
 */
export class Place implements Node {
    /** Number of tokens currently in this place */
    token: number;

    /** Position on the canvas (x, y coordinates) */
    position: Point;

    /** Unique identifier, e.g., 'p1', 'p2' */
    id: string;

    /** Optional display name, e.g., 'Coffee Beans' */
    label?: string;

    constructor(token: number, position: Point, id: string,
    label?: string) {
        this.token = token;
        this.position = position;
        this.id = id;
        this.label = label;
    }

    /**
     * Check if this place has at least the required number of
    tokens.
     * Used for determining if connected transitions can fire.
     */
    hasEnoughTokens(required: number): boolean {
        return this.token >= required;
    }

    /**
     * Consume tokens from this place (called when a transition
    fires).
```

```
38      */
39     removeTokens(count: number): void {
40         this.token = Math.max(0, this.token - count);
41     }
42 
43     /**
44      * Add tokens to this place (called when a transition fires
       ).
45      */
46     addTokens(count: number): void {
47         this.token += count;
48     }
49 }
```

Listing 4.1: Place class definition (src/app/tr-classes/petri-net/place.ts)

4.4 Transitions: Action Nodes

Transitions are represented as rectangles. They model events or actions that can occur when their input conditions are met. Transitions do not hold tokens; instead, they **fire** (execute) when enabled.

```
1  import { Point } from './point';
2  import { Node } from 'src/app/tr-interfaces/petri-net/node';
3  import { Arc } from './arc';
4  
5  /**
6   * Represents a Transition in a Petri net.
7   * Transitions model events/actions and connect input places to
       output places.
8   */
9  export class Transition implements Node {
10     /** Position on the canvas */
11     position: Point;
12 
13     /** Unique identifier, e.g., 't1', 't2' */
14     id: string;
15 
16     /** Optional display name, e.g., 'Grind Beans' */
17     label?: string;
18 
19     /** Arcs coming INTO this transition (from input places) */
20     preArcs: Arc[] = [];
21 
22     /** Arcs going OUT OF this transition (to output places) */
23     postArcs: Arc[] = [];
24 
25     constructor(position: Point, id: string, label?: string) {
26         this.position = position;
27         this.id = id;
28         this.label = label;
29     }
```

```
30
31      /**
32       * Check if this transition is enabled (can fire).
33       * A transition is enabled when ALL input places have
    enough tokens.
34       */
35      isEnabled(): boolean {
36          return this.preArcs.every(arc => {
37              const place = arc.from as Place;
38              return place.hasEnoughTokens(Math.abs(arc.weight));
39          });
40      }
41
42      /**
43       * Fire this transition: consume tokens from inputs,
    produce tokens in outputs.
44       */
45      fire(): void {
46          if (!this.isEnabled()) {
47              console.warn(`Transition ${this.id} cannot fire:
    not enabled`);
48              return;
49          }
50
51          // Consume tokens from input places
52          this.preArcs.forEach(arc => {
53              const place = arc.from as Place;
54              place.removeTokens(Math.abs(arc.weight));
55          });
56
57          // Produce tokens in output places
58          this.postArcs.forEach(arc => {
59              const place = arc.to as Place;
60              place.addTokens(Math.abs(arc.weight));
61          });
62      }
63
64      /** Add an incoming arc (from a place to this transition)
    */
65      appendPreArc(arc: Arc): void {
66          this.preArcs.push(arc);
67      }
68
69      /** Add an outgoing arc (from this transition to a place)
    */
70      appendPostArc(arc: Arc): void {
71          this.postArcs.push(arc);
72      }
73  }
```

Listing 4.2: Transition class definition (src/app/tr-classes/petri-net/transition.ts)

4.4.1 Firing Rules

A transition t is **enabled** when all input places have sufficient tokens:

$$\forall p \in \bullet t : M(p) \geq W(p,t)$$

When an enabled transition fires:

1. Tokens are **consumed** from each input place according to the arc weight
2. Tokens are **produced** in each output place according to the arc weight

4.5 Arcs: Flow Connections

Arcs are directed edges connecting places to transitions or transitions to places. They define the **flow** of tokens through the net.

- **Place** $\rightarrow$ **Transition**: Tokens from the place are required for the transition to fire
- **Transition** $\rightarrow$ **Place**: When the transition fires, tokens are produced in the place

```
import { Node } from 'src/app/tr-interfaces/petri-net/node';
import { Point } from './point';

/**
 * Represents an Arc (directed edge) connecting nodes in a Petri net.
 * Arcs always connect a Place to a Transition or a Transition to a Place.
 */
export class Arc {
    /** Starting node (either Place or Transition) */
    from: Node;

    /** Ending node (either Place or Transition) */
    to: Node;

    /**
     * Arc weight: how many tokens are moved.
     * Default is 1. A weight of 2 means 2 tokens are consumed/produced.
     */
    weight: number;

    /**
     * Optional anchor points for curved arcs.
     * Allows visual bending of the arc on the canvas.
```

```
     */
    anchors: Point[];

    constructor(from: Node, to: Node, weight: number = 1,
    anchors: Point[] = []) {
        this.from = from;
        this.to = to;
        this.weight = weight;
        this.anchors = anchors;
    }

    /**
     * Get the source position for drawing.
     */
    getSourcePosition(): Point {
        return this.from.position;
    }

    /**
     * Get the target position for drawing.
     */
    getTargetPosition(): Point {
        return this.to.position;
    }
}
```

Listing 4.3: Arc class definition (src/app/tr-classes/petri-net/arc.ts)

4.6 The Node Interface

Both `Place` and `Transition` implement the `Node` interface, which defines their common properties:

```
import { Point } from 'src/app/tr-classes/petri-net/point';

/**
 * Common interface for all Petri net nodes (Places and
   Transitions).
 * Enables arcs to connect any type of node generically.
 */
export interface Node {
    /** Position on the canvas */
    position: Point;

    /** Unique identifier within the net */
    id: string;

    /** Optional human-readable label */
    label?: string;
}
```

Listing 4.4: Node interface (src/app/tr-interfaces/petri-net/node.ts)

This interface allows the `Arc` class to work with both places and transitions without knowing the specific type.

4.7 The Point Class

All elements use the `Point` class to define their canvas coordinates:

```
/**
 * Represents a 2D coordinate on the canvas.
 */
export class Point {
    constructor(
        public x: number,
        public y: number,
    ) {}

    /**
     * Calculate distance to another point.
     */
    distanceTo(other: Point): number {
        const dx = this.x - other.x;
        const dy = this.y - other.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    /**
     * Create a copy of this point.
     */
    clone(): Point {
        return new Point(this.x, this.y);
    }
}
```

Listing 4.5: Point class (src/app/tr-classes/petri-net/point.ts)

4.8 Practical Example: Building a Simple Net

Let's create a simple "sending a letter" Petri net programmatically:

```
import { Place } from 'src/app/tr-classes/petri-net/place';
import { Transition } from 'src/app/tr-classes/petri-net/
    transition';
import { Arc } from 'src/app/tr-classes/petri-net/arc';
import { Point } from 'src/app/tr-classes/petri-net/point';

// 1. Create places (states)
const unstampedLetter = new Place(
    1,                      // Initial tokens: 1 letter ready
    new Point(100, 100),    // Position on canvas
    'p1',                   // Unique ID
    'Unstamped Letter'      // Label
```

```
);

const stampedLetter = new Place(
    0,                      // No tokens initially
    new Point(300, 100),
    'p2',
    'Stamped Letter'
);

const mailedLetter = new Place(
    0,
    new Point(500, 100),
    'p3',
    'Mailed Letter'
);

// 2. Create transitions (actions)
const stampAction = new Transition(
    new Point(200, 100),
    't1',
    'Stamp Letter'
);

const mailAction = new Transition(
    new Point(400, 100),
    't2',
    'Mail Letter'
);

// 3. Connect with arcs
const arc1 = new Arc(unstampedLetter, stampAction);  // p1 ->
    t1
const arc2 = new Arc(stampAction, stampedLetter);    // t1 ->
    p2
const arc3 = new Arc(stampedLetter, mailAction);     // p2 ->
    t2
const arc4 = new Arc(mailAction, mailedLetter);      // t2 ->
    p3

// 4. Register arcs with transitions
stampAction.appendPreArc(arc1);
stampAction.appendPostArc(arc2);
mailAction.appendPreArc(arc3);
mailAction.appendPostArc(arc4);

// 5. Check which transitions are enabled
console.log('Stamp enabled:', stampAction.isEnabled());  //
    true (p1 has token)
console.log('Mail enabled:', mailAction.isEnabled());    //
    false (p2 has no token)

// 6. Fire the stamp transition
stampAction.fire();
console.log('After stamping:');
```

```
60 console.log('  p1 tokens:', unstampedLetter.token);  // 0
61 console.log('  p2 tokens:', stampedLetter.token);    // 1
62 console.log('  Mail enabled:', mailAction.isEnabled()); // true
```

Listing 4.6: Creating a Petri net in code

Output:

```
Stamp enabled: true
Mail enabled: false
After stamping:
  p1 tokens: 0
  p2 tokens: 1
  Mail enabled: true
```

4.9 Class Diagram

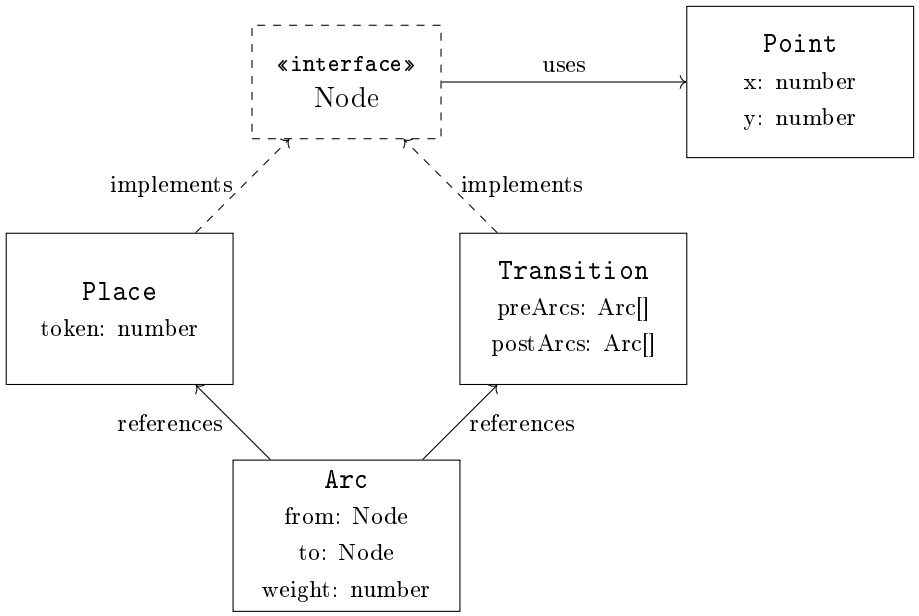

Figure 4.1: Petri net element class relationships

4.10 Summary

The **Petri Net Elements** are the fundamental data objects that define any Petri net:

- **Place**: Holds tokens, represents states/conditions

- **Transition**: Represents actions, fires when enabled
- **Arc**: Connects places and transitions, defines token flow
- **Point**: Stores canvas coordinates for positioning
- **Node interface**: Common contract for places and transitions

These objects are the "things" you manipulate when designing a Petri net. But how does the application manage a *collection* of these elements? How does it track all places, transitions, and arcs in your entire net? That's the role of the **Data Service**, which we explore in the next chapter.

Chapter 5

Data Service: Central Data Store

The previous chapter introduced the **Petri Net Elements**—the individual building blocks (places, transitions, arcs) of our models. This chapter focuses on the **Data Service**, which acts as the central repository for all these elements and ensures the entire application stays synchronized.

5.1 Problem Statement

Consider what happens when you **add a new Place** to your Petri net:

1. You drag a "Place" icon onto the canvas
2. The application creates a new `Place` object in memory
3. This new place must be **stored** somewhere accessible
4. The **drawing component** must know about it to render it
5. The **save function** must include it when exporting
6. The **simulation engine** must consider it when calculating enabled transitions

Without a central data store, every component would maintain its own list of elements. If one component adds a place, how do the others know? If one deletes an arc, how do the others update? This would lead to:

- **Inconsistent state**: Different components showing different data
- **Complex synchronization**: Manual coordination between components

- **Bug-prone code**: Easy to forget updating one of many lists

The `DataService` solves this by being the **single source of truth** for all Petri net data.

5.2 Conceptual Overview

Think of the `DataService` as the application's **central brain** or **shared whiteboard**. It maintains:

- **All Places**: Their tokens, positions, IDs, and labels
- **All Transitions**: Their positions, IDs, labels, and connected arcs
- **All Arcs**: Source/target nodes, weights, and anchor points
- **All Actions**: Unique labels used by transitions

When anything changes, the `DataService`:

1. Updates its internal records
2. **Broadcasts** the change to all interested components

This ensures every part of the application sees the same, up-to-date data.

5.3 Implementation

```
import { Injectable } from '@angular/core';
import { Subject, Observable } from 'rxjs';
import { Arc } from '../tr-classes/petri-net/arc';
import { Place } from '../tr-classes/petri-net/place';
import { Transition } from '../tr-classes/petri-net/transition'
    ;

/**
 * Payload emitted when data changes.
 * fitContent: if true, the UI should re-center the view on the
     net.
 */
interface DataChangedPayload {
    fitContent: boolean;
}

@Injectable({
    providedIn: 'root', // Singleton: one instance for entire
    application
})
export class DataService {
```

```
19      //
        ============================================================================

20      // PRIVATE DATA STORAGE
21      //
        ============================================================================

22
23      /** All places in the current Petri net */
24      private _places: Place[] = [];
25
26      /** All transitions in the current Petri net */
27      private _transitions: Transition[] = [];
28
29      /** All arcs connecting places and transitions */
30      private _arcs: Arc[] = [];
31
32      /** Unique action labels used by transitions */
33      private _actions: string[] = [];
34
35      //
        ============================================================================

36      // CHANGE NOTIFICATION
37      //
        ============================================================================

38
39      /**
40       * Subject for broadcasting data changes.
41       * Components subscribe to be notified when the Petri net
        is modified.
42       */
43      private dataChangedSubject = new Subject<DataChangedPayload
        >();
44
45      /** Public observable for subscribing to data changes */
46      public dataChanged$: Observable<DataChangedPayload> =
47          this.dataChangedSubject.asObservable();
48
49      //
        ============================================================================

50      // GETTERS: Read current data
51      //
        ============================================================================

52
53      getPlaces(): Place[] { return this._places; }
54      getTransitions(): Transition[] { return this._transitions;
        }
55      getArcs(): Arc[] { return this._arcs; }
56      getActions(): string[] { return this._actions; }
57
```

```
58      //
        ============================================================================

59      // SETTERS: Replace entire collections (used when loading
        files)
60      //
        ============================================================================

61
62      set places(value: Place[]) { this._places = value; }
63      set transitions(value: Transition[]) { this._transitions =
        value; }
64      set arcs(value: Arc[]) { this._arcs = value; }
65      set actions(value: string[]) { this._actions = value; }
66
67      //
        ============================================================================

68      // MODIFICATION METHODS
69      //
        ============================================================================

70
71      /**
72       * Add a new place to the Petri net.
73       */
74      addPlace(place: Place): void {
75          this._places.push(place);
76          this.triggerDataChanged();
77      }
78
79      /**
80       * Remove a place and all its connected arcs.
81       */
82      removePlace(deletablePlace: Place): void {
83          // First, remove all arcs connected to this place
84          const connectedArcs = this._arcs.filter(
85              arc => arc.from === deletablePlace || arc.to ===
        deletablePlace
86          );
87          connectedArcs.forEach(arc => this.removeArc(arc));
88
89          // Then remove the place itself
90          this._places = this._places.filter(place => place !==
        deletablePlace);
91
92          this.triggerDataChanged();
93      }
94
95      /**
96       * Add a new transition to the Petri net.
97       */
98      addTransition(transition: Transition): void {
99          this._transitions.push(transition);
```

```
100         this.triggerDataChanged();
101     }
102
103     /**
104      * Remove a transition and all its connected arcs.
105      */
106     removeTransition(deletableTransition: Transition): void {
107         // Remove connected arcs
108         const connectedArcs = this._arcs.filter(
109             arc => arc.from === deletableTransition || arc.to
    === deletableTransition
110         );
111         connectedArcs.forEach(arc => this.removeArc(arc));
112
113         // Remove the transition
114         this._transitions = this._transitions.filter(t => t !==
     deletableTransition);
115
116         this.triggerDataChanged();
117     }
118
119     /**
120      * Connect two nodes with a new arc.
121      * Validates that the connection follows Petri net rules:
122      * - Place -> Transition (input arc)
123      * - Transition -> Place (output arc)
124      */
125     connectNodes(from: Node, to: Node): Arc | null {
126         // Validate: cannot connect Place to Place or
    Transition to Transition
127         if (from instanceof Place && to instanceof Place) {
128             console.error('Cannot connect Place to Place');
129             return null;
130         }
131         if (from instanceof Transition && to instanceof
    Transition) {
132             console.error('Cannot connect Transition to
    Transition');
133             return null;
134         }
135
136         const arc = new Arc(from, to, 1);
137         this._arcs.push(arc);
138
139         // Register arc with the transition
140         if (from instanceof Place && to instanceof Transition)
    {
141             to.appendPreArc(arc);  // Input arc
142         } else if (from instanceof Transition && to instanceof
    Place) {
143             from.appendPostArc(arc);  // Output arc
144         }
145
146         this.triggerDataChanged();
```

```
147         return arc;
148     }
149 
150     /**
151      * Remove an arc from the Petri net.
152      */
153     removeArc(arc: Arc): void {
154         this._arcs = this._arcs.filter(a => a !== arc);
155 
156         // Also remove from transition's arc lists
157         if (arc.to instanceof Transition) {
158             arc.to.preArcs = arc.to.preArcs.filter(a => a !==
    arc);
159         }
160         if (arc.from instanceof Transition) {
161             arc.from.postArcs = arc.from.postArcs.filter(a => a
     !== arc);
162         }
163     }
164 
165     //
    ============================================================================

166     // UTILITY METHODS
167     //
    ============================================================================

168 
169     /**
170      * Find a place by its ID.
171      */
172     getPlaceById(id: string): Place | undefined {
173         return this._places.find(p => p.id === id);
174     }
175 
176     /**
177      * Find a transition by its ID.
178      */
179     getTransitionById(id: string): Transition | undefined {
180         return this._transitions.find(t => t.id === id);
181     }
182 
183     /**
184      * Check if the Petri net is empty.
185      */
186     isEmpty(): boolean {
187         return this._places.length === 0 && this._transitions.
    length === 0;
188     }
189 
190     /**
191      * Clear all data (reset to empty net).
192      */
193     clearAll(): void {
```

```
194         this._places = [];
195         this._transitions = [];
196         this._arcs = [];
197         this._actions = [];
198         this.triggerDataChanged(true);
199     }
200
201     //
    ============================================================================

202     // CHANGE NOTIFICATION
203     //
    ============================================================================


204
205     /**
206      * Broadcast that data has changed.
207      * All subscribed components will be notified.
208      *
209      * @param fitContent If true, UI should re-center the view
210      */
211     public triggerDataChanged(fitContent: boolean = false):
    void {
212         console.log('[DataService] Data changed, fitContent:',
    fitContent);
213         this.dataChangedSubject.next({ fitContent });
214     }
215 }
```

Listing 5.1: DataService core implementation (src/app/tr-services/data.service.ts)

5.4 Key Concepts

5.4.1 Singleton Pattern

The `@Injectable({providedIn: 'root'})` decorator ensures only **one instance** of `DataService` exists. All components access the same instance, guaranteeing they work with the same data.

5.4.2 Subject vs. BehaviorSubject

Unlike the `UiService` which uses `BehaviorSubject` (remembers the last value), the `DataService` uses a plain `Subject`:

- **Subject**: Only emits to current subscribers; new subscribers don't receive past events
- **Why?** Components don't need the "last change event"—they need the *current data*, which they get via `getPlaces()`, `getTransitions()`, etc.

5.4.3 Cascading Deletes

When removing a place or transition, all connected arcs are automatically removed:

```
removePlace(deletablePlace: Place): void {
    // 1. Find connected arcs
    const connectedArcs = this._arcs.filter(
        arc => arc.from === deletablePlace || arc.to === deletablePlace
    );

    // 2. Remove each connected arc
    connectedArcs.forEach(arc => this.removeArc(arc));

    // 3. Remove the place itself
    this._places = this._places.filter(place => place !== deletablePlace);

    // 4. Notify subscribers
    this.triggerDataChanged();
}
```

Listing 5.2: Cascading delete pattern

5.5 Usage Patterns

5.5.1 Adding Elements

```
import { Component } from '@angular/core';
import { DataService } from 'src/app/tr-services/data.service';
import { Place } from 'src/app/tr-classes/petri-net/place';
import { Point } from 'src/app/tr-classes/petri-net/point';

@Component({ /* ... */ })
export class PetriNetComponent {
    constructor(private dataService: DataService) {}

    /**
     * Called when user clicks on canvas with "Add Place" tool selected.
     */
    onCanvasClick(x: number, y: number): void {
        // Generate unique ID
        const newId = `p${this.dataService.getPlaces().length + 1}`;

        // Create the place object
        const newPlace = new Place(0, new Point(x, y), newId);

        // Add to DataService (automatically notifies subscribers)
```

```
        this.dataService.addPlace(newPlace);
    }
}
```

Listing 5.3: Adding a place via DataService

5.5.2 Subscribing to Changes

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subscription } from 'rxjs';
import { DataService } from 'src/app/tr-services/data.service';

@Component({ /* ... */ })
export class DrawingCanvasComponent implements OnInit,
    OnDestroy {
    private dataSubscription: Subscription;

    constructor(private dataService: DataService) {}

    ngOnInit(): void {
        // Subscribe to data changes
        this.dataSubscription = this.dataService.dataChanged$.
    subscribe(payload => {
            console.log('Data changed! Redrawing...');

            // Get latest data
            const places = this.dataService.getPlaces();
            const transitions = this.dataService.getTransitions
    ();
            const arcs = this.dataService.getArcs();

            // Redraw the canvas
            this.redraw(places, transitions, arcs);

            // Optionally re-center the view
            if (payload.fitContent) {
                this.fitContentToViewport();
            }
        });

        // Initial draw
        this.redraw(
            this.dataService.getPlaces(),
            this.dataService.getTransitions(),
            this.dataService.getArcs()
        );
    }

    ngOnDestroy(): void {
        this.dataSubscription?.unsubscribe();
    }

```

```
42      private redraw(places: Place[], transitions: Transition[],
        arcs: Arc[]): void {
43          // Clear and redraw SVG elements...
44      }
45  }
```

Listing 5.4: Subscribing to data changes

5.6 Data Flow Visualization

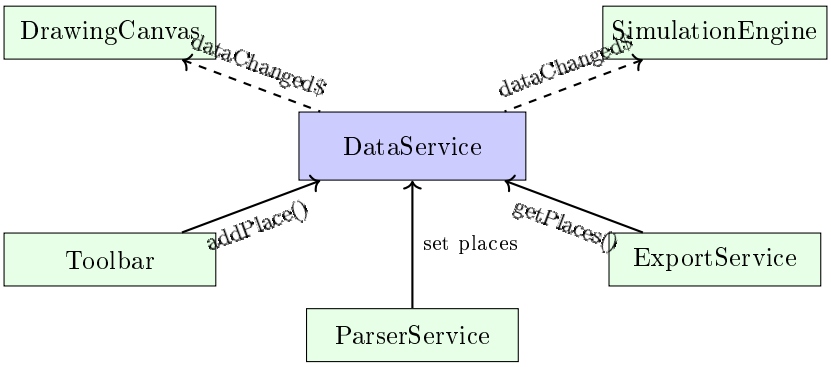

Figure 5.1: DataService as central hub for Petri net data

The diagram shows:

- **Solid arrows**: Components that *modify* data (Toolbar adds elements, ParserService loads files)
- **Dashed arrows**: Components that *subscribe* to changes (Canvas redraws, SimulationEngine recalculates)
- **Query arrows**: Components that *read* data on demand (ExportService for saving)

5.7 Interaction with Other Services

The `DataService` is used by nearly every other service:

5.8 Summary

The `DataService` is the application's **single source of truth** for Petri net data:

- **Stores** all places, transitions, arcs, and actions

Service	Interaction
`ParserService`	Loads data from JSON/PNML files
`PnmlService`	Imports/exports PNML format
`ExportJsonDataService`	Exports to custom JSON format
`LayoutSugiyamaService`	Modifies element positions
`ZoomService`	Reads element positions for bounds calculation
`SimulationService`	Reads transitions for firing sequence

Table 5.1: Services that interact with DataService

- **Provides** getter methods for reading current state
- **Offers** modification methods with automatic cascading (e.g., deleting connected arcs)
- **Broadcasts** changes via `dataChanged$` observable
- **Enables** decoupled architecture where components don't directly communicate

With the core data model and storage understood, the next chapter explores how users interact with the canvas through **Zoom and Pan** functionality.

Chapter 6

System Architecture

The previous chapters introduced the core building blocks: the `UiService` for managing UI state (Chapter 3), the Petri net elements as data objects (Chapter 4), and the `DataService` as the central data store (Chapter 5). This chapter provides a high-level overview of how these components fit together within the overall system architecture.

6.1 System Overview

The L3S-Offshore-2 system follows a client-server architecture with clear separation between the visualization frontend and the simulation backend.

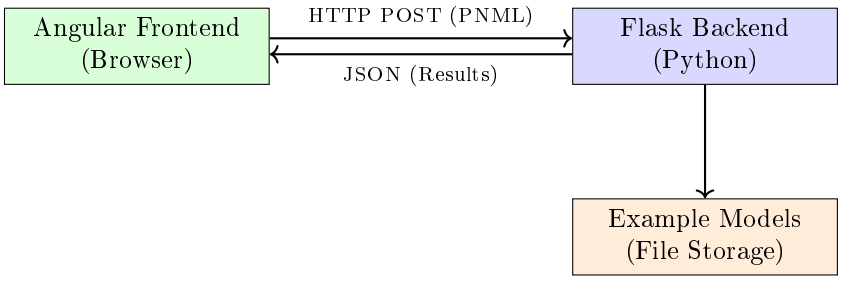

Figure 6.1: High-level system architecture showing frontend-backend interaction

6.1.1 Frontend (Angular)

The frontend is a single-page application (SPA) responsible for:

- Rendering Petri nets as interactive SVG graphics
- Handling user interactions (editing, navigation, simulation control)
- Managing application state across different tabs

- Communicating with the backend via REST API

6.1.2 Backend (Flask)

The Python backend provides:

- PNML parsing and validation
- Petri net simulation using the `gspn4py` library
- Offshore scheduling algorithms
- Example model storage and retrieval

6.2 Frontend Component Hierarchy

The Angular application is organized into a hierarchical component structure that reflects the UI layout.

6.2.1 Root Components

AppComponent The root component that bootstraps the application and contains the main layout

HeaderComponent Displays the project banner with L3S branding

FooterComponent Shows institutional logos with links to L3S and LUH homepages

6.2.2 Tab Components

The main content area uses Angular Material's tab group:

ButtonBarComponent The central coordinator that manages tab switching and toolbar rendering for each tab (Build, Code, Simulation, Analyze, Save, Offshore)

PetriNetComponent Renders the interactive SVG visualization of the Petri net

CodeEditorComponent Provides a text editor for direct PNML manipulation

ParameterTableComponent Displays and edits simulation parameters

OffshoreViewComponent Specialized view for offshore scheduling parameters

6.2.3 Dialog Components

Modal dialogs for user feedback and interaction:

DataPersistenceInfoDialogComponent Warns users about data loss on page reload

ErrorPopupComponent Displays error messages from backend or parsing failures

HelpPopupComponent Provides contextual help

TransitionFiringInfoPopupComponent Shows firing statistics for transitions

6.3 Service Layer

Services are singleton classes that provide shared functionality and state management. The most important services are covered in dedicated chapters.

6.3.1 Core Services

DataService Central state container for the Petri net model. Manages places, transitions, arcs, and the current marking. Emits change events via RxJS observables. *See Chapter 5 for details.*

UiService Manages UI state including the current tab, simulation mode (automatic/manual), selected elements, and zoom level. *See Chapter 3 for details.*

ZoomService Handles viewport transformations (pan, zoom) with ResizeObserver integration for responsive behavior. *See Section 7.1 for details.*

ParserService Converts between PNML XML, internal data structures, and the JSON representation used by the backend. *See Chapter 8 for details.*

6.3.2 Simulation Services

SimulationService Communicates with the backend's simulation endpoints. Manages firing sequences and multi-run results.

TokenGameService Implements the manual simulation mode where users can step through the firing sequence or click on enabled transitions.

PlanningService Handles the offshore scheduling API calls and parameter management. *See Chapter 10 for details.*

6.3.3 Utility Services

ExportImageService Exports the current view as PNG

ExportSvgService Exports the Petri net as SVG

ExportJsonDataService Exports the internal representation as JSON. *See Chapter 8 for details.*

LayoutSugiyamaService Implements the Sugiyama algorithm for automatic layout. *See Chapter 9 for details.*

6.4 Data Flow

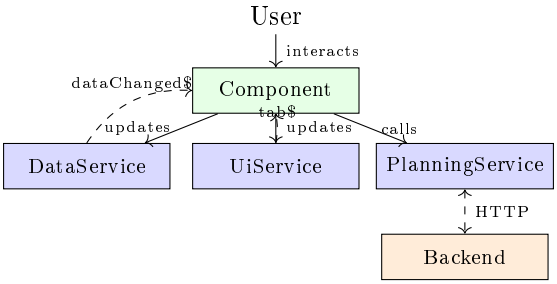

Figure 6.2: Data flow between components and services

The application follows a unidirectional data flow pattern:

1. **User Input:** User interacts with UI (clicks, drags, edits)
2. **Component Handler:** Component method processes the event
3. **Service Update:** Component calls service method to update state
4. **Observable Emission:** Service emits new state via RxJS Subject
5. **Subscriber Notification:** All subscribed components receive update
6. **View Re-render:** Angular change detection updates the DOM

6.5 Tab State Management

A key architectural feature is the tab-aware state management that ensures UI consistency when switching between tabs.

6.5.1 The Tab$ Observable

The `UiService` exposes a `tab$` BehaviorSubject that emits the current tab name whenever the user navigates:

```
private _tab$ = new BehaviorSubject<string>('build');

get tab$(): Observable<string> {
  return this._tab$.asObservable();
}

set tab(value: string) {
  console.log(`[UiService] Tab changed: ${this._tab$.value} ->
    ${value}`);
  this._tab$.next(value);
}
```

Listing 6.1: Tab state observable in UiService

6.5.2 Tab Guards in Components

Components subscribe to `tab$` and conditionally enable/disable features:

```
this.uiService.tab$.subscribe(tab => {
  if (tab !== 'simulation') {
    this.clearAllHighlights();
    this.stopAnimationTimer();
  } else {
    this.restoreSimulationState();
  }
});
```

Listing 6.2: Tab-aware highlighting in PetriNetComponent

Chapter 7

Implementation Details

This chapter describes the key implementation aspects of the frontend, focusing on the most complex and interesting technical solutions.

7.1 Zoom and Pan System

One of the most technically challenging aspects was implementing a robust zoom and pan system that behaves intuitively across different viewport sizes.

7.1.1 The Problem

The original fork had a basic zoom implementation that exhibited several issues:

- Content would "drift" away from the center when zooming
- Mouse coordinates were incorrect after panning
- Elements would not be placed at the click position in the Build tab
- The "Fit Content" feature would not trigger correctly after loading a file

7.1.2 Viewport-Centered Zooming

The solution involved centering all zoom operations around the viewport center rather than the origin:

```
zoomAtCenter(factor: number): void {
  const viewportCenterX = this.viewportWidth / 2;
  const viewportCenterY = this.viewportHeight / 2;

  // Transform viewport center to SVG coordinates
  const svgX = (viewportCenterX - this.panX) / this.scale;
  const svgY = (viewportCenterY - this.panY) / this.scale;
```

```
8
9   // Apply new scale
10  this.scale *= factor;
11  this.scale = Math.max(0.1, Math.min(5, this.scale));
12
13  // Adjust pan to keep the same SVG point at viewport center
14  this.panX = viewportCenterX - svgX * this.scale;
15  this.panY = viewportCenterY - svgY * this.scale;
16 }
```

Listing 7.1: Viewport-centered zoom calculation

7.1.3 ResizeObserver Integration

To handle viewport size changes (e.g., when switching from the Code tab's 50% width back to full width), a `ResizeObserver` was integrated:

```
1  private resizeObserver: ResizeObserver;
2  private shouldAutoFit = false;
3
4  ngAfterViewInit(): void {
5    this.resizeObserver = new ResizeObserver(entries => {
6      for (const entry of entries) {
7        this.zoomService.updateViewportDimensions(
8          entry.contentRect.width,
9          entry.contentRect.height
10       );
11       if (this.shouldAutoFit) {
12         this.fitContentToViewport();
13       }
14     }
15   });
16   this.resizeObserver.observe(this.svgContainer.nativeElement);
17 }
```

Listing 7.2: ResizeObserver setup for responsive viewport

7.2 Simulation Animation

The simulation animation system displays the firing sequence received from the backend.

7.2.1 Firing Sequence Format

The backend returns a firing sequence as an array of transition IDs:

```
1  interface SimulationResult {
2    firing_seq: string[];      // e.g., ["t1", "t3", "t2", "t1"]
3    detailed_log: StepLog[];   // Optional: token counts at each
       step
4  }
```

Listing 7.3: Firing sequence data structure

7.2.2 Animation Loop

The animation uses `setInterval` with a configurable speed:

```
private animationTimer: any;

startAutoplay(): void {
  const interval = 1000 / this.playbackSpeed;
  this.animationTimer = setInterval(() => {
    if (this.currentStep < this.firingSequence.length) {
      this.highlightTransition(this.firingSequence[this.
    currentStep]);
      this.currentStep++;
      this.timelinePosition = this.currentStep;
    } else {
      this.stopAutoplay();
    }
  }, interval);
}
```

Listing 7.4: Animation timer implementation

7.2.3 Transition Highlighting

Transitions are highlighted by applying CSS classes that trigger SVG animations:

```
highlightTransition(transitionId: string): void {
  // Remove previous highlight
  this.clearHighlights();

  // Find the SVG element
  const element = document.getElementById(`transition-${
    transitionId}`);
  if (element) {
    element.classList.add('firing');
    // CSS handles the animation:
    // .firing { fill: #ff6600; filter: drop-shadow(0 0 8px #
    ff6600); }
  }
}
```

Listing 7.5: Transition highlighting with CSS

7.3 Token Reset Logic

A critical feature is resetting tokens to the initial marking when leaving the Simulation tab.

7.3.1 Baseline Snapshot

When the user first enters the Simulation tab, a snapshot of the current marking is saved:

```
private baselineMarking: Map<string, number> = new Map();

saveBaselineMarking(): void {
  this.dataService.places.forEach(place => {
    this.baselineMarking.set(place.id, place.tokens);
  });
}

resetTokensToBaseline(): void {
  this.baselineMarking.forEach((tokens, placeId) => {
    const place = this.dataService.getPlaceById(placeId);
    if (place) {
      place.tokens = tokens;
    }
  });
  this.dataService.triggerDataChanged();
}
```

Listing 7.6: Baseline marking snapshot

7.3.2 Tab-Aware Reset

The reset is triggered when navigating away from the Simulation tab:

```
this.uiService.tab$.subscribe(newTab => {
  if (this.previousTab === 'simulation' && newTab !== '
    simulation') {
    this.resetTokensToBaseline();
    this.clearAllHighlights();
  }
  if (newTab === 'simulation' && this.previousTab !== '
    simulation') {
    this.saveBaselineMarking();
  }
  this.previousTab = newTab;
});
```

Listing 7.7: Tab-aware token reset

7.4 Parameter Table

The parameter table allows users to configure simulation parameters before running.

7.4.1 Dynamic Form Generation

Parameters are dynamically generated from the backend's DTO definition:

```
generateFormControls(parameters: ParameterDefinition[]): void {
  parameters.forEach(param => {
    this.formGroup.addControl(
      param.name,
      new FormControl(param.defaultValue, this.getValidators(param))
    );
  });
}
```

Listing 7.8: Dynamic parameter form generation

7.4.2 Reactive Forms Integration

The initial implementation caused severe UI freezes due to incorrect parent-child form integration. The solution used Angular's `ControlContainer`:

```
@Component({
  selector: 'app-parameter-row',
  viewProviders: [
    { provide: ControlContainer, useExisting: FormGroupDirective }
  ]
})
export class ParameterRowComponent {
  // Child component now correctly integrates with parent form
}
```

Listing 7.9: ControlContainer fix for nested forms

7.5 Right-Click Statistics

To avoid conflicts with the manual simulation mode (left-click fires transition), statistics are displayed via right-click.

```
@HostListener('contextmenu', ['$event'])
onRightClick(event: MouseEvent): void {
  if (this.uiService.tab !== 'simulation') return;

  event.preventDefault(); // Suppress browser context menu

  const transitionId = this.getTransitionIdFromEvent(event);
  if (transitionId) {
    const stats = this.simulationService.getTransitionStats(transitionId);
    this.showStatisticsPopup(event.clientX, event.clientY, stats);
  }
```

```
}
```

Listing 7.10: Right-click handler for statistics popup

7.6 Data Persistence Warning

Users are warned about data loss through two mechanisms:

7.6.1 Initial Dialog

A dialog appears on first visit (controlled by `localStorage`):

```
ngOnInit(): void {
  const shown = localStorage.getItem('dataPersistenceInfoShown');
  if (!shown) {
    this.dialog.open(DataPersistenceInfoDialogComponent);
  }
}
```

Listing 7.11: Data persistence dialog on app start

7.6.2 Beforeunload Guard

The browser's native confirmation dialog is triggered when closing with unsaved data:

```
@HostListener('window:beforeunload', ['$event'])
onBeforeUnload(event: BeforeUnloadEvent): void {
  if (!this.dataService.isEmpty()) {
    event.preventDefault();
    event.returnValue = ''; // Required for Chrome
  }
}
```

Listing 7.12: Beforeunload event handler

Chapter 8

PNML and JSON Data I/O

This chapter covers the application's data persistence capabilities: how Petri nets are saved to files and loaded from files. The I/O services act as translators between external file formats and the internal data structures.

8.1 Problem Statement

After spending time building a Petri net, users need to:

- **Save** their work for later continuation
- **Share** models with colleagues
- **Load** existing models from other tools

Without I/O capabilities, all work would be lost when closing the browser. The application supports two file formats to address different needs.

8.2 Supported Formats

Feature	PNML	Custom JSON
Format type	XML-based	JSON
Standard	ISO/IEC 15909-2	Application-specific
Complexity	Verbose, detailed	Concise, simple
Interoperability	WoPeD, CPN Tools, etc.	This application only
Human-readable	Moderate	High

Table 8.1: Comparison of supported file formats

8.3 Custom JSON Format

The custom JSON format is designed for simplicity and ease of parsing:

```typescript
/**
 * Structure of our custom JSON format for Petri nets.
 */
export interface JsonPetriNet {
    /** List of place IDs, e.g., ["p1", "p2", "p3"] */
    places: string[];

    /** List of transition IDs, e.g., ["t1", "t2"] */
    transitions: string[];

    /** Arc definitions as "source,target": weight pairs */
    arcs?: { [idPair: string]: number };  // e.g., "p1,t1": 1

    /** Unique action labels for transitions */
    actions?: string[];

    /** Transition labels as id: label pairs */
    labels?: { [transitionId: string]: string };

    /** Initial marking as id: tokenCount pairs */
    marking?: { [placeId: string]: number };

    /** Element positions as id: {x, y} pairs */
    layout?: { [id: string]: Coords | Coords[] };
}

export interface Coords {
    x: number;
    y: number;
}
```

Listing 8.1: JSON format interface (src/app/classes/json-petri-net.ts)

8.3.1 Example JSON File

```json
{
  "places": ["p1", "p2", "p3"],
  "transitions": ["t1", "t2"],
  "arcs": {
    "p1,t1": 1,
    "t1,p2": 1,
    "p2,t2": 1,
    "t2,p3": 1
  },
  "marking": {
    "p1": 1,
    "p2": 0,
    "p3": 0
  },
```

```
  "labels": {
    "t1": "Stamp Letter",
    "t2": "Mail Letter"
  },
  "layout": {
    "p1": {"x": 100, "y": 100},
    "t1": {"x": 200, "y": 100},
    "p2": {"x": 300, "y": 100},
    "t2": {"x": 400, "y": 100},
    "p3": {"x": 500, "y": 100}
  }
}
```

Listing 8.2: Example Petri net in JSON format

8.4 Exporting to JSON

The `ExportJsonDataService` converts the internal data model to JSON format:

```
import { Injectable } from '@angular/core';
import { DataService } from './data.service';
import { JsonPetriNet, Coords } from '../classes/json-petri-net
    ';

@Injectable({ providedIn: 'root' })
export class ExportJsonDataService {
    constructor(private dataService: DataService) {}

    /**
     * Export the current Petri net as a JSON file download.
     */
    public exportAsJson(): void {
        // 1. Build the JsonPetriNet structure
        const jsonObj = this.buildJsonStructure();

        // 2. Serialize to formatted JSON string
        const jsonString = JSON.stringify(jsonObj, null, 2);

        // 3. Trigger browser download
        this.downloadFile(jsonString, 'petri-net.json', '
    application/json');
    }

    private buildJsonStructure(): JsonPetriNet {
        const json: JsonPetriNet = {
            places: [],
            transitions: [],
            arcs: {},
            marking: {},
            labels: {},
            layout: {},
```

```
        };

        // Collect places
        for (const place of this.dataService.getPlaces()) {
            json.places.push(place.id);
            json.layout![place.id] = { x: place.position.x, y:
    place.position.y };

            if (place.token > 0) {
                json.marking![place.id] = place.token;
            }
        }

        // Collect transitions
        for (const transition of this.dataService.
    getTransitions()) {
            json.transitions.push(transition.id);
            json.layout![transition.id] = {
                x: transition.position.x,
                y: transition.position.y
            };

            if (transition.label) {
                json.labels![transition.id] = transition.label;
            }
        }

        // Collect arcs
        for (const arc of this.dataService.getArcs()) {
            const key = `${arc.from.id},${arc.to.id}`;
            json.arcs![key] = arc.weight;

            // Store anchor points if present
            if (arc.anchors.length > 0) {
                json.layout![key] = arc.anchors.map(p => ({ x:
    p.x, y: p.y }));
            }
        }

        return json;
    }

    private downloadFile(content: string, filename: string,
    mimeType: string): void {
        const blob = new Blob([content], { type: mimeType });
        const url = URL.createObjectURL(blob);

        const link = document.createElement('a');
        link.href = url;
        link.download = filename;
        link.click();

        URL.revokeObjectURL(url);
    }
```

```
}
```

Listing 8.3: JSON export service (src/app/tr-services/export-json-data.service.ts)

8.5 Importing from JSON

The `ParserService` converts JSON files back to internal objects:

```
import { Injectable } from '@angular/core';
import { JsonPetriNet, Coords } from '../classes/json-petri-net';
import { Place } from '../tr-classes/petri-net/place';
import { Transition } from '../tr-classes/petri-net/transition';
import { Arc } from '../tr-classes/petri-net/arc';
import { Point } from '../tr-classes/petri-net/point';

@Injectable({ providedIn: 'root' })
export class ParserService {
    /** Flag: true if some elements had no position data */
    incompleteLayoutData: boolean = false;

    /**
     * Parse a JSON string into Petri net elements.
     * Returns arrays of places, transitions, arcs, and actions.
     */
    parse(jsonText: string): [Place[], Transition[], Arc[], string[]] {
        this.incompleteLayoutData = false;

        // 1. Parse JSON string to object
        const data: JsonPetriNet = JSON.parse(jsonText);

        const places: Place[] = [];
        const transitions: Transition[] = [];
        const arcs: Arc[] = [];
        const actions: string[] = data.actions || [];

        // 2. Create Place objects
        for (const placeId of data.places) {
            const position = this.getPosition(data, placeId);
            const tokens = data.marking?.[placeId] || 0;
            const label = data.labels?.[placeId];

            places.push(new Place(tokens, position, placeId, label));
        }

        // 3. Create Transition objects
        for (const transitionId of data.transitions) {
```

```
39              const position = this.getPosition(data,
    transitionId);
40              const label = data.labels?.[transitionId];
41
42              transitions.push(new Transition(position,
    transitionId, label));
43          }
44
45          // 4. Create Arc objects and link to nodes
46          if (data.arcs) {
47              for (const [idPair, weight] of Object.entries(data.
    arcs)) {
48                  const [fromId, toId] = idPair.split(',');
49
50                  const fromNode = this.findNode(fromId, places,
    transitions);
51                  const toNode = this.findNode(toId, places,
    transitions);
52
53                  if (fromNode && toNode) {
54                      const anchors = this.getAnchors(data,
    idPair);
55                      const arc = new Arc(fromNode, toNode,
    weight, anchors);
56                      arcs.push(arc);
57
58                      // Register with transition
59                      if (toNode instanceof Transition) {
60                          toNode.appendPreArc(arc);
61                      }
62                      if (fromNode instanceof Transition) {
63                          fromNode.appendPostArc(arc);
64                      }
65                  }
66              }
67          }
68
69          return [places, transitions, arcs, actions];
70      }
71
72      /**
73       * Get position from layout data, or return default (0,0).
74       */
75      private getPosition(data: JsonPetriNet, id: string): Point
    {
76          if (data.layout && data.layout[id]) {
77              const coords = data.layout[id] as Coords;
78              return new Point(coords.x, coords.y);
79          }
80
81          this.incompleteLayoutData = true;
82          return new Point(0, 0);
83      }
84
```

```
    /**
     * Get arc anchor points from layout data.
     */
    private getAnchors(data: JsonPetriNet, arcId: string): Point[] {
        if (data.layout && Array.isArray(data.layout[arcId])) {
            return (data.layout[arcId] as Coords[]).map(c => new Point(c.x, c.y));
        }
        return [];
    }

    /**
     * Find a node (place or transition) by ID.
     */
    private findNode(id: string, places: Place[], transitions: Transition[]): Place | Transition | undefined {
        return places.find(p => p.id === id) || transitions.find(t => t.id === id);
    }
}
```

Listing 8.4: JSON parser service (src/app/tr-services/parser.service.ts)

8.6 PNML Format

PNML (Petri Net Markup Language) is an XML-based international standard:

```
<?xml version="1.0" encoding="UTF-8"?>
<pnml>
  <net id="net1" type="http://www.pnml.org/version-2009/grammar/ptnet">
    <page id="page1">
      <place id="p1">
        <name><text>Unstamped Letter</text></name>
        <graphics>
          <position x="100" y="100"/>
        </graphics>
        <initialMarking>
          <text>1</text>
        </initialMarking>
      </place>

      <transition id="t1">
        <name><text>Stamp Letter</text></name>
        <graphics>
          <position x="200" y="100"/>
        </graphics>
      </transition>

      <arc id="a1" source="p1" target="t1">
```

```
        <inscription><text>1</text></inscription>
      </arc>
    </page>
  </net>
</pnml>
```

Listing 8.5: Example PNML document structure

8.7 PNML Import and Export

The `PnmlService` handles bidirectional PNML conversion:

```
import { Injectable } from '@angular/core';
import { xml2js } from 'xml-js';
import { DataService } from './data.service';
import { Place } from '../tr-classes/petri-net/place';
import { Transition } from '../tr-classes/petri-net/transition';
import { Arc } from '../tr-classes/petri-net/arc';
import { Point } from '../tr-classes/petri-net/point';

@Injectable({ providedIn: 'root' })
export class PnmlService {
    incompleteLayoutData: boolean = false;

    constructor(
        private dataService: DataService,
        private layoutService: LayoutSugiyamaService
    ) {}

    /**
     * Parse PNML XML string and load into DataService.
     */
    parse(xmlString: string): [Place[], Transition[], Arc[], string[]] {
        this.incompleteLayoutData = false;

        // 1. Convert XML to JavaScript object
        const xmlObj = xml2js(xmlString, { compact: false });

        // 2. Find all place, transition, and arc elements
        const pnmlPlaces = this.findElements(xmlObj, 'place');
        const pnmlTransitions = this.findElements(xmlObj, 'transition');
        const pnmlArcs = this.findElements(xmlObj, 'arc');

        // 3. Convert to internal objects
        const places = this.parsePlaces(pnmlPlaces);
        const transitions = this.parseTransitions(pnmlTransitions);
        const arcs = this.parseArcs(pnmlArcs, places, transitions);
```

```
36          const actions = this.extractActions(transitions);
37
38          // 4. Update DataService
39          this.dataService.places = places;
40          this.dataService.transitions = transitions;
41          this.dataService.arcs = arcs;
42          this.dataService.actions = actions;
43
44          // 5. Apply automatic layout if positions were missing
45          if (this.incompleteLayoutData) {
46              this.layoutService.applySugiyamaLayout();
47          }
48
49          this.dataService.triggerDataChanged(true);
50
51          return [places, transitions, arcs, actions];
52      }
53
54      /**
55       * Parse a PNML place element into a Place object.
56       */
57      private parsePlace(element: any): Place {
58          const id = element.attributes.id;
59
60          // Extract name from nested <name><text> elements
61          const nameElement = this.findChild(element, 'name');
62          const label = nameElement ? this.getTextContent(
    nameElement) : undefined;
63
64          // Extract position from <graphics><position>
65          const position = this.extractPosition(element);
66
67          // Extract initial marking from <initialMarking><text>
68          const markingElement = this.findChild(element, '
    initialMarking');
69          const tokens = markingElement ? parseInt(this.
    getTextContent(markingElement)) || 0 : 0;
70
71          return new Place(tokens, position, id, label);
72      }
73
74      /**
75       * Extract position from graphics element, or mark as
    incomplete.
76       */
77      private extractPosition(element: any): Point {
78          const graphics = this.findChild(element, 'graphics');
79          const position = graphics ? this.findChild(graphics, '
    position') : null;
80
81          if (position?.attributes?.x && position?.attributes?.y)
     {
82              return new Point(
83                  parseFloat(position.attributes.x),
```

```
                parseFloat(position.attributes.y)
            );
        }

        this.incompleteLayoutData = true;
        return new Point(0, 0);
    }

    // ... additional helper methods ...
}
```

Listing 8.6: PNML service (src/app/tr-services/pnml.service.ts) - Import

8.7.1 PNML Export

```
/**
 * Generate PNML XML string from current DataService content.
 */
public getPNML(): string {
    const places = this.dataService.getPlaces();
    const transitions = this.dataService.getTransitions();
    const arcs = this.dataService.getArcs();

    // Build XML structure
    let xml = `<?xml version="1.0" encoding="UTF-8"?>
<pnml>
  <net id="net1" type="http://www.pnml.org/version-2009/grammar
    /ptnet">
    <page id="page1">
`;

    // Add places
    for (const place of places) {
        xml += this.placeToXml(place);
    }

    // Add transitions
    for (const transition of transitions) {
        xml += this.transitionToXml(transition);
    }

    // Add arcs
    for (const arc of arcs) {
        xml += this.arcToXml(arc);
    }

    xml += `    </page>
  </net>
</pnml>`;

    return this.formatXml(xml);
}
```

```
37
38 private placeToXml(place: Place): string {
39     return `      <place id="${place.id}">
40         <name><text>${place.label || ''}</text></name>
41         <graphics>
42           <position x="${place.position.x}" y="${place.position
    .y}"/>
43         </graphics>
44         <initialMarking><text>${place.token}</text></
    initialMarking>
45       </place>
46 `;
47 }
48
49 private transitionToXml(transition: Transition): string {
50     return `      <transition id="${transition.id}">
51         <name><text>${transition.label || ''}</text></name>
52         <graphics>
53           <position x="${transition.position.x}" y="${
    transition.position.y}"/>
54         </graphics>
55       </transition>
56 `;
57 }
58
59 private arcToXml(arc: Arc): string {
60     return `      <arc id="arc-${arc.from.id}-${arc.to.id}"
    source="${arc.from.id}" target="${arc.to.id}">
61         <inscription><text>${arc.weight}</text></inscription>
62       </arc>
63 `;
64 }
```

Listing 8.7: PNML export method

8.8 Import Flow Visualization

8.9 Handling Missing Layout Data

When loading files without position information (common with PNML files from other tools), the services set `incompleteLayoutData = true`. The calling code then automatically applies the Sugiyama layout algorithm (covered in the next chapter) to generate readable positions.

8.10 Summary

The I/O services enable data persistence and interoperability:

- **ExportJsonDataService**: Exports to custom JSON format

- **ParserService**: Imports from custom JSON format
- **PnmlService**: Bidirectional PNML (XML) conversion
- **Automatic layout**: Applied when position data is missing
- **Download trigger**: Browser's download API for file saving

The next chapter covers the **Layout Algorithms** that automatically arrange Petri net elements into readable configurations.

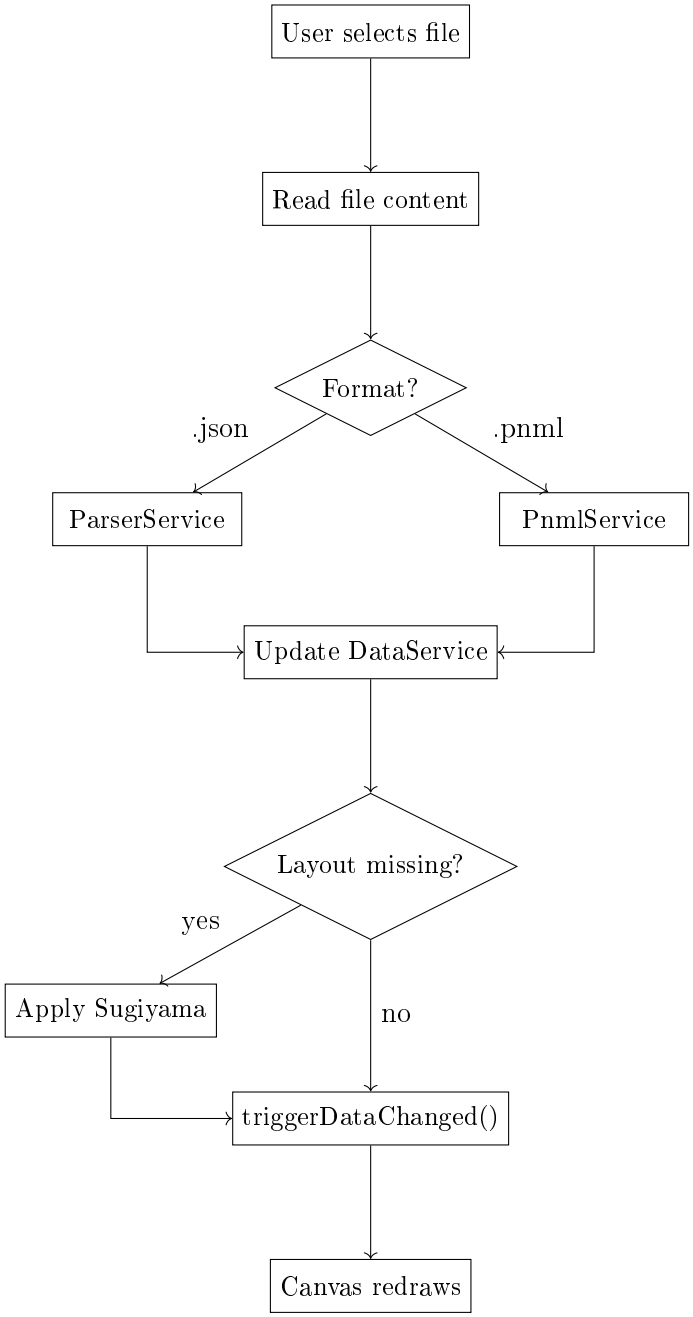

Figure 8.1: File import processing flow

Chapter 9

Layout Algorithms

When loading a Petri net without position information, or after extensive manual editing, the diagram can become cluttered and difficult to read. This chapter covers the automatic layout algorithms that transform messy diagrams into clear, structured visualizations.

9.1 Problem Statement

Consider importing a PNML file from another tool that stored no graphical information. All elements would be placed at position $(0, 0)$, creating an unusable overlap. Even with positions, diagrams can suffer from:

- **Overlapping elements**: Places and transitions stacked on each other
- **Arc crossings**: Lines crossing unnecessarily, obscuring the flow
- **Poor spacing**: Elements too close or too far apart
- **No visual hierarchy**: The logical flow of the process is not apparent

Layout algorithms solve these problems by calculating optimal positions for all elements.

9.2 Available Algorithms

The application provides two layout algorithms with different characteristics:

9.3 Spring Embedder Algorithm

The Spring Embedder models the Petri net as a physical system:

- **Nodes** act as electrically charged particles that **repel** each other

Algorithm	Approach	Best For
Spring Embedder	Physics simulation	General, organic layouts
Sugiyama	Hierarchical layering	Sequential processes, flowcharts

Table 9.1: Layout algorithm comparison

- **Arcs** act as springs that **attract** connected nodes toward an ideal length

The algorithm iteratively moves nodes until forces reach equilibrium.

9.3.1 Force Calculation

For each node, the total force is the sum of:

1. **Repulsion** from all other nodes: $F_{rep} = \frac{C_{rep}}{d^2}$
2. **Spring attraction/repulsion** from connected nodes: $F_{spring} = C_{spring} \cdot (d - L)$

Where d is the distance between nodes, L is the ideal arc length, and C_{rep}, C_{spring} are constants.

```
import { Injectable } from '@angular/core';
import { DataService } from './data.service';
import { Point } from '../tr-classes/petri-net/point';
import { Node } from '../tr-interfaces/petri-net/node';

@Injectable({ providedIn: 'root' })
export class LayoutSpringEmbedderService {
    // Algorithm parameters
    private epsilon = 0.01;        // Minimum force for
    termination
    private maxIterations = 5000; // Maximum iteration count
    private cRep = 20000;          // Repulsion constant
    private idealLength = 150;     // Ideal arc length
    private cSpring = 20;          // Spring constant

    constructor(private dataService: DataService) {}

    /**
     * Apply Spring Embedder layout to all nodes.
     * Runs asynchronously with visual feedback.
     */
    async layoutSpringEmbedder(): Promise<void> {
        const nodes: Node[] = [
            ...this.dataService.getPlaces(),
            ...this.dataService.getTransitions()
        ];

```

```
27          // Initialize random positions if all at origin
28          this.initializePositions(nodes);
29
30          let maxForce = this.epsilon + 1;
31          let iteration = 0;
32
33          // Iterate until equilibrium or max iterations
34          while (iteration < this.maxIterations && maxForce >
    this.epsilon) {
35              const forces: Map<string, Point> = new Map();
36
37              // Calculate forces for each node
38              for (const node of nodes) {
39                  const force = this.calculateTotalForce(node,
    nodes);
40                  forces.set(node.id, force);
41              }
42
43              // Apply forces and track maximum
44              maxForce = 0;
45              for (const node of nodes) {
46                  const force = forces.get(node.id)!;
47                  node.position.x += force.x;
48                  node.position.y += force.y;
49
50                  const magnitude = Math.sqrt(force.x ** 2 +
    force.y ** 2);
51                  maxForce = Math.max(maxForce, magnitude);
52              }
53
54              // Yield for UI updates (every 10 iterations)
55              if (iteration % 10 === 0) {
56                  this.dataService.triggerDataChanged();
57                  await this.sleep(1);
58              }
59
60              iteration++;
61          }
62
63          console.log(`Spring Embedder completed in ${iteration}
    iterations`);
64          this.dataService.triggerDataChanged(true);
65      }
66
67      /**
68       * Calculate total force on a node from all other nodes.
69       */
70      private calculateTotalForce(node: Node, allNodes: Node[]):
    Point {
71          let fx = 0, fy = 0;
72
73          for (const other of allNodes) {
74              if (other.id === node.id) continue;
75
```

```
76              // Repulsion force (always present)
77              const repulsion = this.calculateRepulsion(node.
    position, other.position);
78              fx += repulsion.x;
79              fy += repulsion.y;
80
81              // Spring force (only for connected nodes)
82              if (this.areConnected(node, other)) {
83                  const spring = this.calculateSpring(node.
    position, other.position);
84                  fx += spring.x;
85                  fy += spring.y;
86              }
87          }
88
89          return new Point(fx, fy);
90      }
91
92      /**
93       * Calculate repulsion force between two positions.
94       * Inversely proportional to distance squared.
95       */
96      private calculateRepulsion(p1: Point, p2: Point): Point {
97          const dx = p1.x - p2.x;
98          const dy = p1.y - p2.y;
99          const distance = Math.sqrt(dx * dx + dy * dy);
100
101         // Prevent division by zero
102         if (distance < 0.1) {
103             return new Point(
104                 (Math.random() - 0.5) * 10,
105                 (Math.random() - 0.5) * 10
106             );
107         }
108
109         const factor = this.cRep / (distance * distance);
110         return new Point(
111             (dx / distance) * factor,
112             (dy / distance) * factor
113         );
114     }
115
116     /**
117      * Calculate spring force between connected nodes.
118      * Attracts if too far, repels if too close.
119      */
120     private calculateSpring(p1: Point, p2: Point): Point {
121         const dx = p2.x - p1.x;
122         const dy = p2.y - p1.y;
123         const distance = Math.sqrt(dx * dx + dy * dy);
124
125         if (distance < 0.1) return new Point(0, 0);
126
127         // Displacement from ideal length
```

```
128         const displacement = distance - this.idealLength;
129         const factor = this.cSpring * displacement / distance;
130
131         return new Point(dx * factor, dy * factor);
132     }
133
134     private sleep(ms: number): Promise<void> {
135         return new Promise(resolve => setTimeout(resolve, ms));
136     }
137
138     // ... helper methods ...
139 }
```

Listing 9.1: Spring Embedder implementation (src/app/tr-services/layout-spring-embedder.service.ts)

9.4 Sugiyama Algorithm

The Sugiyama algorithm [7] creates hierarchical, layered layouts ideal for sequential processes. It operates in four phases:

1. **Cycle Removal**: Temporarily reverse some arcs to create a DAG
2. **Layer Assignment**: Assign nodes to horizontal layers
3. **Vertex Ordering**: Order nodes within layers to minimize crossings
4. **Coordinate Assignment**: Calculate final x, y positions

9.4.1 Phase 1: Cycle Removal

Petri nets often contain cycles (feedback loops). For hierarchical layout, cycles must be temporarily broken:

```
1  export class CycleRemovalService {
2      private reversedArcs: Arc[] = [];
3
4      constructor(private nodes: Node[], private arcs: Arc[]) {}
5
6      /**
7       * Remove cycles by reversing back-edges.
8       * Uses depth-first search to detect cycles.
9       */
10     removeCycles(): void {
11         const visited = new Set<string>();
12         const recursionStack = new Set<string>();
13
14         for (const node of this.nodes) {
15             if (!visited.has(node.id)) {
16                 this.dfs(node, visited, recursionStack);
17             }
```

```
        }
    }

    private dfs(node: Node, visited: Set<string>, stack: Set<
    string>): void {
        visited.add(node.id);
        stack.add(node.id);

        const outgoingArcs = this.arcs.filter(a => a.from.id
    === node.id);

        for (const arc of outgoingArcs) {
            const targetId = arc.to.id;

            if (stack.has(targetId)) {
                // Back-edge detected: reverse this arc
                this.reverseArc(arc);
                this.reversedArcs.push(arc);
            } else if (!visited.has(targetId)) {
                this.dfs(arc.to, visited, stack);
            }
        }

        stack.delete(node.id);
    }

    /**
     * Restore reversed arcs after layout is complete.
     */
    reverseArcs(): void {
        for (const arc of this.reversedArcs) {
            this.reverseArc(arc);
        }
    }

    private reverseArc(arc: Arc): void {
        const temp = arc.from;
        arc.from = arc.to;
        arc.to = temp;
    }
}
```

Listing 9.2: Cycle removal service

9.4.2 Phase 2: Layer Assignment

Nodes are assigned to layers based on their distance from source nodes:

```
export class LayerAssignmentService {
    constructor(private nodes: Node[], private arcs: Arc[]) {}

    /**
```

```
     * Assign each node to a layer using longest-path algorithm.
     * Returns array of layers, each containing nodes at that level.
     */
    assignLayers(): Node[][] {
        const layers: Node[][] = [];
        const nodeLayer = new Map<string, number>();

        // Find source nodes (no incoming arcs)
        const sources = this.nodes.filter(n =>
            !this.arcs.some(a => a.to.id === n.id)
        );

        // Assign sources to layer 0
        for (const source of sources) {
            nodeLayer.set(source.id, 0);
        }

        // BFS to assign remaining nodes
        const queue = [...sources];
        while (queue.length > 0) {
            const node = queue.shift()!;
            const currentLayer = nodeLayer.get(node.id)!;

            // Find successors
            const outArcs = this.arcs.filter(a => a.from.id === node.id);
            for (const arc of outArcs) {
                const successor = arc.to;
                const newLayer = currentLayer + 1;

                // Update if this path is longer
                if (!nodeLayer.has(successor.id) ||
                    nodeLayer.get(successor.id)! < newLayer) {
                    nodeLayer.set(successor.id, newLayer);
                    queue.push(successor);
                }
            }
        }

        // Build layer arrays
        const maxLayer = Math.max(...nodeLayer.values());
        for (let i = 0; i <= maxLayer; i++) {
            layers[i] = this.nodes.filter(n => nodeLayer.get(n.id) === i);
        }

        return layers;
    }
}
```

Listing 9.3: Layer assignment service

9.4.3 Phase 3: Vertex Ordering

Within each layer, nodes are reordered to minimize arc crossings:

```
export class VertexOrderingService {
    constructor(
        private layers: Node[][],
        private arcs: Arc[]
    ) {}

    /**
     * Order vertices using barycenter heuristic.
     * Minimizes edge crossings between adjacent layers.
     */
    orderVertices(): void {
        // Multiple passes for better results
        for (let pass = 0; pass < 4; pass++) {
            // Forward pass
            for (let i = 1; i < this.layers.length; i++) {
                this.orderLayer(i, i - 1);
            }
            // Backward pass
            for (let i = this.layers.length - 2; i >= 0; i--) {
                this.orderLayer(i, i + 1);
            }
        }
    }

    /**
     * Order nodes in targetLayer based on connections to
    fixedLayer.
     */
    private orderLayer(targetIdx: number, fixedIdx: number):
    void {
        const targetLayer = this.layers[targetIdx];
        const fixedLayer = this.layers[fixedIdx];

        // Calculate barycenter for each node
        const barycenters = targetLayer.map(node => {
            const connectedIndices = this.getConnectedIndices(
    node, fixedLayer);
            if (connectedIndices.length === 0) return Infinity;

            const sum = connectedIndices.reduce((a, b) => a + b
    , 0);
            return sum / connectedIndices.length;
        });

        // Sort by barycenter
        const indexed = targetLayer.map((node, i) => ({ node,
    bc: barycenters[i] }));
        indexed.sort((a, b) => a.bc - b.bc);

        this.layers[targetIdx] = indexed.map(item => item.node)
```

```
    ;
46      }
47
48      private getConnectedIndices(node: Node, layer: Node[]):
    number[] {
49          const indices: number[] = [];
50
51          for (const arc of this.arcs) {
52              let connected: Node | null = null;
53              if (arc.from.id === node.id) connected = arc.to;
54              if (arc.to.id === node.id) connected = arc.from;
55
56              if (connected) {
57                  const idx = layer.findIndex(n => n.id ===
    connected!.id);
58                  if (idx !== -1) indices.push(idx);
59              }
60          }
61
62          return indices;
63      }
64  }
```

Listing 9.4: Vertex ordering (barycenter method)

9.4.4 Phase 4: Coordinate Assignment

Finally, x and y coordinates are calculated based on layers and ordering:

```
1   export class CoordinateAssignmentService {
2       private layerSpacing = 180;  // Horizontal space between
    layers
3       private nodeSpacing = 100;   // Vertical space between
    nodes
4
5       constructor(
6           private layers: Node[][],
7           private canvasWidth: number = 1200,
8           private canvasHeight: number = 600
9       ) {}
10
11      /**
12       * Assign final coordinates to all nodes.
13       */
14      assignCoordinates(): void {
15          const totalWidth = (this.layers.length - 1) * this.
    layerSpacing;
16          const startX = (this.canvasWidth - totalWidth) / 2;
17
18          for (let layerIdx = 0; layerIdx < this.layers.length;
    layerIdx++) {
19              const layer = this.layers[layerIdx];
20              const x = startX + layerIdx * this.layerSpacing;
```

```
21
22              // Center nodes vertically
23              const totalHeight = (layer.length - 1) * this.
    nodeSpacing;
24              const startY = (this.canvasHeight - totalHeight) /
    2;
25
26              for (let nodeIdx = 0; nodeIdx < layer.length;
    nodeIdx++) {
27                  const node = layer[nodeIdx];
28                  const y = startY + nodeIdx * this.nodeSpacing;
29
30                  node.position = new Point(x, y);
31              }
32          }
33      }
34  }
```

Listing 9.5: Coordinate assignment

9.4.5 Orchestrating the Phases

```
1  @Injectable({ providedIn: 'root' })
2  export class LayoutSugiyamaService {
3      constructor(private dataService: DataService) {}
4
5      /**
6       * Apply complete Sugiyama layout algorithm.
7       */
8      applySugiyamaLayout(): void {
9          const nodes = [
10             ...this.dataService.getPlaces(),
11             ...this.dataService.getTransitions()
12         ];
13         const arcs = [...this.dataService.getArcs()];
14
15         // Phase 1: Remove cycles
16         const cycleRemoval = new CycleRemovalService(nodes,
    arcs);
17         cycleRemoval.removeCycles();
18
19         // Phase 2: Assign layers
20         const layerAssignment = new LayerAssignmentService(
    nodes, arcs);
21         const layers = layerAssignment.assignLayers();
22
23         // Restore original arc directions
24         cycleRemoval.reverseArcs();
25
26         // Phase 3: Order vertices
27         const vertexOrdering = new VertexOrderingService(layers
    , arcs);
```

```
28          vertexOrdering.orderVertices();
29
30          // Phase 4: Assign coordinates
31          const coordAssignment = new CoordinateAssignmentService
    (layers);
32          coordAssignment.assignCoordinates();
33
34          // Notify UI
35          this.dataService.triggerDataChanged(true);
36      }
37  }
```

Listing 9.6: Sugiyama layout service (src/app/tr-services/layout-sugiyama.service.ts)

9.5 Visual Comparison

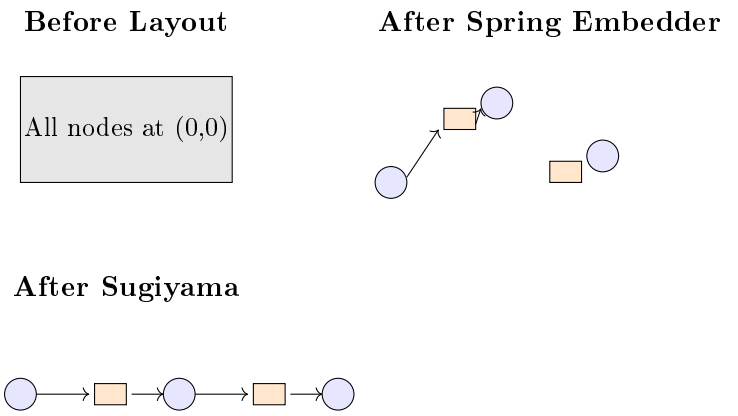

Figure 9.1: Layout algorithm results comparison

9.6 When to Use Which Algorithm

- **Spring Embedder**:
  - Petri nets with many cycles
  - Organic, symmetric patterns
  - When hierarchy is not important
- **Sugiyama**:
  - Sequential, workflow-like processes
  - When you want left-to-right or top-to-bottom flow
  - PNML imports without layout data (default)

9.7 Summary

Layout algorithms transform cluttered diagrams into readable visualizations:

- **Spring Embedder**: Physics-based, produces organic layouts
- **Sugiyama**: Four-phase hierarchical algorithm
  1. Cycle removal (DFS-based)
  2. Layer assignment (longest path)
  3. Vertex ordering (barycenter method)
  4. Coordinate assignment
- Both algorithms modify the `position` property of nodes directly
- `triggerDataChanged(true)` notifies UI to redraw and fit content

The next chapter covers the **Planning Service**, which enables communication with the backend for simulation and analysis.

Chapter 10

Planning Service: Backend API Integration

The previous chapters covered client-side functionality: UI state, data management, file I/O, and layout algorithms. This chapter introduces the **Planning Service**, which extends the application's capabilities by communicating with a powerful backend server for simulation and analysis tasks.

10.1 Problem Statement

Some operations are too complex or resource-intensive for browser-based execution:

- **Long simulations**: Running thousands of firing steps
- **Stochastic analysis**: Monte Carlo simulations with multiple runs
- **AI planning**: Computing optimal transition sequences
- **Domain-specific algorithms**: Offshore scheduling optimization

These tasks require:

- More computing power than browsers provide
- Specialized libraries (e.g., Python's `gspn4py` for stochastic Petri nets)
- Server-side state management for long-running operations

The `PlanningService` acts as a **bridge** between the frontend and backend, sending requests and receiving results via HTTP.

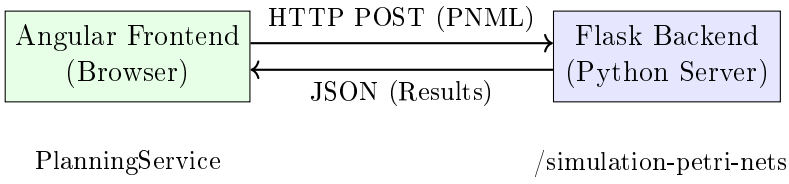

Figure 10.1: Frontend-backend communication via PlanningService

10.2 System Architecture

10.3 Environment Configuration

The backend URL is configured via Angular's environment files:

```
export const environment = {
    production: false,
    backendApiUrl: 'http://localhost:9040/l3s-offshore-2'
};
```

Listing 10.1: Development environment (src/environments/environment.ts)

```
export const environment = {
    production: true,
    backendApiUrl: 'https://l3s-offshore-plan-2.l3s.uni-
    hannover.de/l3s-offshore-2'
};
```

Listing 10.2: Production environment (src/environments/environment.prod.ts)

Angular automatically swaps these files during the build process, ensuring development builds connect to `localhost` while production builds use the live server.

10.4 PlanningService Implementation

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpErrorResponse } from '@angular/common/
    http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { environment } from '../../environments/environment';

@Injectable({
    providedIn: 'root'
})
export class PlanningService {
    /** Base URL for planning-related endpoints */
```

```
12      private apiUrl = `${environment.backendApiUrl}/dto/planning
        `;
13
14      /** URL for Petri net simulation endpoints */
15      private simpleSimApiUrl = `${environment.backendApiUrl}/
        simulation-petri-nets/simple-sim`;
16
17      constructor(private http: HttpClient) {}
18
19      //
        ============================================================================
20      // SIMULATION METHODS
21      //
        ============================================================================
22
23      /**
24       * Run a Petri net simulation on the backend.
25       * Sends the PNML file and receives firing sequence results
        .
26       *
27       * @param pnmlFile - The PNML file to simulate
28       * @param runs - Number of simulation runs (for stochastic
        analysis)
29       * @returns Observable with simulation results
30       */
31      runSimpleSimulation(pnmlFile: File, runs: number = 1):
        Observable<SimulationResult> {
32          // FormData is required for file uploads
33          const formData = new FormData();
34          formData.append('pnml_model', pnmlFile, pnmlFile.name);
35          formData.append('num_runs', runs.toString());
36
37          return this.http.post<SimulationResult>(this.
        simpleSimApiUrl, formData)
38              .pipe(catchError(this.handleError));
39      }
40
41      /**
42       * Convenience method: run simulation from PNML string.
43       * Converts the string to a File object before sending.
44       */
45      runSimpleSimulationFromString(
46          pnmlContent: string,
47          runs: number = 1,
48          fileName: string = 'model.pnml'
49      ): Observable<SimulationResult> {
50          // Create a File from the string content
51          const pnmlFile = new File([pnmlContent], fileName, {
        type: 'application/xml' });
52          return this.runSimpleSimulation(pnmlFile, runs);
53      }
54
```

```
55      //
        ============================================================================

56      // PARAMETER & CONFIGURATION METHODS
57      //
        ============================================================================

58
59      /**
60       * Fetch default parameter values from backend.
61       * Used to populate the parameter table with sensible
        defaults.
62       */
63      getDefaults(): Observable<ParameterDefaults> {
64          return this.http.get<ParameterDefaults>(`${this.apiUrl
        }/defaults`)
65              .pipe(catchError(this.handleError));
66      }
67
68      /**
69       * Submit planning parameters to backend.
70       * Triggers the offshore scheduling optimization.
71       */
72      runPlanning(planningData: PlanningRequest): Observable<
        PlanningResult> {
73          return this.http.post<PlanningResult>(this.apiUrl,
        planningData)
74              .pipe(catchError(this.handleError));
75      }
76
77      //
        ============================================================================

78      // EXAMPLE MODEL METHODS
79      //
        ============================================================================

80
81      /**
82       * Get list of available example models from backend.
83       * Returns filenames like ["example1.pnml", "example2.pnml
        "]
84       */
85      getAvailableExampleModels(): Observable<string[]> {
86          const url = `${environment.backendApiUrl}/simulation-
        petri-nets/example-models`;
87          return this.http.get<string[]>(url)
88              .pipe(catchError(this.handleError));
89      }
90
91      /**
92       * Load a specific example model by name.
93       * Returns the raw PNML content as a string.
94       */
```

```
95      loadExampleModelByName(name: string): Observable<string> {
96          const url = `${environment.backendApiUrl}/simulation-
    petri-nets/example-models/${encodeURIComponent(name)}`;
97          return this.http.get(url, { responseType: 'text' })
98              .pipe(catchError(this.handleError));
99      }
100
101     //
    ============================================================================

102     // ERROR HANDLING
103     //
    ============================================================================

104
105     /**
106      * Centralized error handler for all HTTP requests.
107      * Logs the error and returns a user-friendly Observable
    error.
108      */
109     private handleError(error: HttpErrorResponse): Observable<
    never> {
110         let message: string;
111
112         if (error.error instanceof ErrorEvent) {
113             // Client-side error (network issues, etc.)
114             message = `Network error: ${error.error.message}`;
115         } else {
116             // Server-side error
117             message = `Server error: ${error.status} - ${error.
    message}`;
118         }
119
120         console.error('[PlanningService]', message);
121         return throwError(() => new Error(message));
122     }
123 }
124
125 //
    ============================================================================

126 // TYPE DEFINITIONS
127 //
    ============================================================================

128
129 interface SimulationResult {
130     /** Sequence of transition IDs that fired */
131     firing_seq: string[];
132     /** Token counts at each step (optional) */
133     detailed_log?: StepLog[];
134     /** Statistics for multi-run simulations */
135     statistics?: TransitionStats;
136 }
```

```
137
138 interface StepLog {
139     step: number;
140     transition: string;
141     marking: { [placeId: string]: number };
142 }
143
144 interface TransitionStats {
145     [transitionId: string]: {
146         fire_count: number;
147         fire_percentage: number;
148     };
149 }
150
151 interface ParameterDefaults {
152     [paramName: string]: number | string | boolean;
153 }
154
155 interface PlanningRequest {
156     model_pnml: string;
157     parameters: { [key: string]: any };
158 }
159
160 interface PlanningResult {
161     schedule: any[];
162     metrics: { [key: string]: number };
163 }
```

Listing 10.3: Planning service (src/app/tr-services/planning.service.ts)

10.5 Usage in Components

10.5.1 Running a Simulation

```
1  import { Component } from '@angular/core';
2  import { PlanningService } from '../../tr-services/planning.
       service';
3  import { PnmlService } from '../../tr-services/pnml.service';
4  import { UiService } from '../../tr-services/ui.service';
5
6  @Component({ /* ... */ })
7  export class ButtonBarComponent {
8      constructor(
9          private planningService: PlanningService,
10         private pnmlService: PnmlService,
11         private uiService: UiService
12     ) {}
13
14     /**
15      * Called when user clicks "Run Simulation" button.
16      */
17     runSimulation(): void {
```

```
18          // 1. Get current Petri net as PNML string
19          const pnmlContent = this.pnmlService.getPNML();
20
21          // 2. Send to backend for simulation
22          this.planningService
23              .runSimpleSimulationFromString(pnmlContent, 1)
24              .subscribe({
25                  next: (result) => {
26                      console.log('Simulation complete:', result.
    firing_seq.length, 'steps');
27
28                      // 3. Update UI with results
29                      this.uiService.setSimulationResults(result)
    ;
30                      this.uiService.tab = TabState.Simulation;
31                  },
32                  error: (err) => {
33                      console.error('Simulation failed:', err.
    message);
34                      this.showErrorDialog('Simulation failed: '
    + err.message);
35                  }
36              });
37      }
38  }
```

Listing 10.4: Using PlanningService in ButtonBarComponent

10.5.2 Loading Example Models

```
1  import { Component, OnInit } from '@angular/core';
2  import { Observable } from 'rxjs';
3  import { PlanningService } from '../../tr-services/planning.
       service';
4  import { PnmlService } from '../../tr-services/pnml.service';
5
6  @Component({
7      selector: 'app-example-selector',
8      template: `
9          <select (change)="onModelSelected($event)">
10             <option value="">-- Select Example --</option>
11             <option *ngFor="let model of availableModels$ |
       async" [value]="model">
12                 {{ model }}
13             </option>
14         </select>
15     `
16 })
17 export class ExampleSelectorComponent implements OnInit {
18     availableModels$: Observable<string[]>;
19
20     constructor(
```

```
21          private planningService: PlanningService,
22          private pnmlService: PnmlService
23      ) {}
24
25      ngOnInit(): void {
26          // Fetch available examples on component init
27          this.availableModels$ = this.planningService.
    getAvailableExampleModels();
28      }
29
30      onModelSelected(event: Event): void {
31          const select = event.target as HTMLSelectElement;
32          const modelName = select.value;
33
34          if (!modelName) return;
35
36          // Load the selected model
37          this.planningService.loadExampleModelByName(modelName).
    subscribe({
38              next: (pnmlContent) => {
39                  // Parse and load into DataService
40                  this.pnmlService.parse(pnmlContent);
41                  console.log('Loaded example:', modelName);
42              },
43              error: (err) => {
44                  console.error('Failed to load example:', err.
    message);
45              }
46          });
47      }
48  }
```

Listing 10.5: Loading example models with dropdown

10.6 Request/Response Flow

10.7 Backend API Endpoints

The backend exposes the following REST endpoints:

Method	Endpoint	Description
GET	`/dto/planning/defaults`	Get default parameter values
POST	`/dto/planning`	Run planning optimization
POST	`/simulation-petri-nets/simple-sim`	Run Petri net simulation
GET	`/simulation-petri-nets/example-models`	List available examples
GET	`/simulation-petri-nets/example-models/:name`	Get example PNML content

Table 10.1: Backend API endpoints

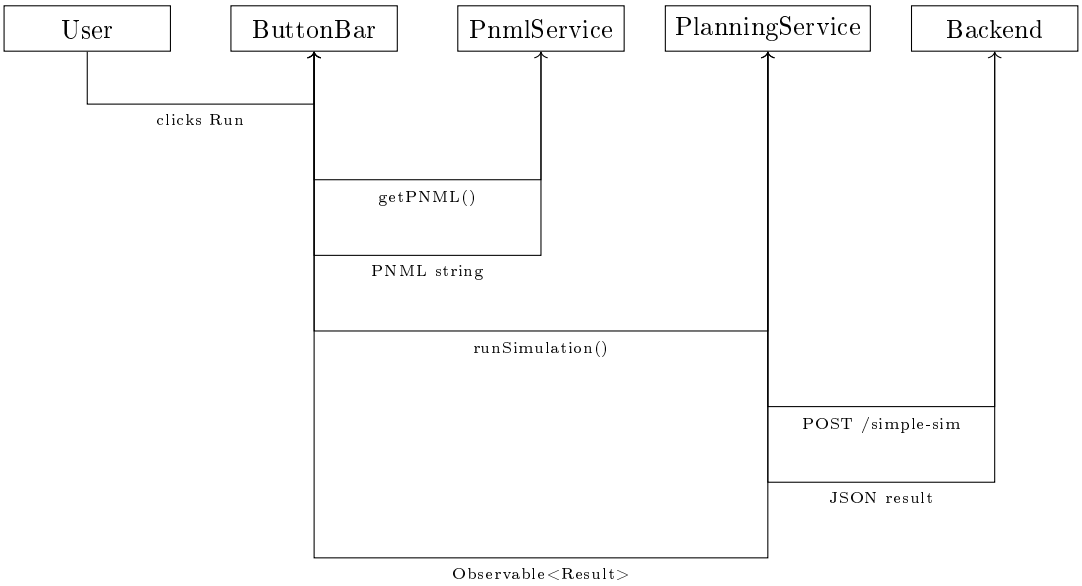

Figure 10.2: Simulation request flow through PlanningService

10.8 Error Handling Patterns

Robust error handling ensures graceful degradation when the backend is unavailable:

```
import { catchError, retry, timeout } from 'rxjs/operators';
import { of } from 'rxjs';

// With retry and timeout
this.planningService.runSimpleSimulation(file)
    .pipe(
        timeout(30000),  // 30 second timeout
        retry(2),        // Retry twice on failure
        catchError(err => {
            // Fallback to empty result
            console.warn('Simulation failed, using empty result');
            return of({ firing_seq: [], detailed_log: [] });
        })
    )
    .subscribe(result => {
        // Handle result (real or fallback)
    });
```

Listing 10.6: Error handling with retry and fallback

10.9 CORS Configuration

For development, the backend must allow cross-origin requests from the Angular dev server:

```python
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app, resources={
    r"/l3s-offshore-2/*": {
        "origins": ["http://localhost:4200"],
        "methods": ["GET", "POST", "OPTIONS"],
        "allow_headers": ["Content-Type"]
    }
})
```

Listing 10.7: Flask CORS configuration (backend)

In production, the nginx reverse proxy handles CORS headers.

10.10 Summary

The `PlanningService` extends frontend capabilities through backend communication:

- **HTTP Communication**: Uses Angular's `HttpClient` for REST API calls
- **Environment Configuration**: Separate URLs for development and production
- **File Uploads**: Uses `FormData` to send PNML files
- **Error Handling**: Centralized handler with user-friendly messages
- **Observable Pattern**: Async results via RxJS Observables

This service enables the application to leverage powerful server-side algorithms while maintaining a responsive, client-side user interface.

Chapter 11

Deployment

This chapter describes how to build and deploy the L3S-Offshore-2 Frontend, both locally for development and on production servers.

11.1 Prerequisites

11.1.1 Development Environment

The following software is required for local development:

- **Node.js:** Version 18.x or higher (LTS recommended)
- **npm:** Version 9.x or higher (comes with Node.js)
- **Angular CLI:** Version 17.x (`npm install -g @angular/cli`)
- **Git:** For version control

11.1.2 Production Environment

For production deployment:

- **Docker:** Version 20.x or higher
- **Docker Compose:** (optional, for multi-container setups)
- **nginx:** Included in the Docker image

11.2 Local Development

11.2.1 Clone and Install

```
# Clone the repository
git clone https://github.com/krausality/PNML_Frontend.git
cd PNML_Frontend

# Install dependencies
npm install
```

Listing 11.1: Cloning and installing dependencies

11.2.2 Development Server

Start the Angular development server with hot reloading:

```
# Start development server
ng serve

# Or with explicit host binding (for network access)
ng serve --host 0.0.0.0 --port 4200
```

Listing 11.2: Starting the development server

The application will be available at http://localhost:4200.

11.2.3 Backend Connection

The development environment expects the Flask backend to run on port 9040. Configure the backend URL in `src/environments/environment.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:9040'
};
```

Listing 11.3: Development environment configuration

11.3 Production Build

11.3.1 Angular Production Build

Create an optimized production build:

```
# Production build with optimizations
ng build --configuration production

# Output is generated in dist/pnml-frontend/
```

Listing 11.4: Creating a production build

The production build includes:

- Ahead-of-Time (AOT) compilation

- Tree shaking to remove unused code
- Minification and uglification
- Content hashing for cache busting

11.3.2 Build Verification

Verify the build output:

```
# Check bundle sizes
ls -la dist/pnml-frontend/browser/

# Expected output: main.js, polyfills.js, styles.css
# Total size should be under 500KB gzipped
```

Listing 11.5: Verifying the production build

11.4 Docker Deployment

11.4.1 Dockerfile

The application uses a multi-stage Dockerfile:

```
# Stage 1: Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve with nginx
FROM nginx:alpine
COPY --from=builder /app/dist/pnml-frontend/browser /usr/share/
    nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 11.6: Multi-stage Dockerfile

11.4.2 Nginx Configuration

The nginx configuration handles SPA routing:

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;
```

```
6
7       # SPA fallback: serve index.html for all routes
8       location / {
9           try_files $uri $uri/ /index.html;
10      }
11
12      # Cache static assets
13      location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$
        {
14          expires 1y;
15          add_header Cache-Control "public, immutable";
16      }
17
18      # Gzip compression
19      gzip on;
20      gzip_types text/plain text/css application/json application
        /javascript;
21  }
```

Listing 11.7: nginx.conf for SPA routing

11.4.3 Building the Docker Image

```
1  # Build the image
2  docker build -t l3s-offshore-frontend:latest .
3
4  # Run the container
5  docker run -d -p 8080:80 --name offshore-frontend l3s-offshore-
       frontend:latest
6
7  # Verify it's running
8  curl http://localhost:8080
```

Listing 11.8: Building and running the Docker container

11.5 Server Deployment

11.5.1 L3S Infrastructure

The L3S research infrastructure uses a nested VPS architecture. Access requires:

1. SSH to the gateway server (`gate.l3s.uni-hannover.de`)
2. SSH to the project VPS
3. Docker commands within the VPS

11.5.2 Deployment Script

A typical deployment workflow:

```
#!/bin/bash
# deploy.sh

# 1. Build and push to container registry
docker build -t registry.l3s.de/offshore/frontend:latest .
docker push registry.l3s.de/offshore/frontend:latest

# 2. On the server: pull and restart
ssh user@server << 'EOF'
  docker pull registry.l3s.de/offshore/frontend:latest
  docker stop offshore-frontend || true
  docker rm offshore-frontend || true
  docker run -d -p 80:80 --name offshore-frontend \
    registry.l3s.de/offshore/frontend:latest
EOF
```

Listing 11.9: Production deployment script

11.5.3 Docker Cleanup

After deployment, clean up old images to save disk space:

```
# Remove unused images (not just containers)
docker image prune -a --filter "until=24h"

# Remove stopped containers
docker container prune

# Full cleanup (use with caution)
docker system prune -a
```

Listing 11.10: Docker cleanup commands

11.6 Environment Configuration

11.6.1 Production Environment

Update `src/environments/environment.prod.ts` with production URLs:

```
export const environment = {
  production: true,
  apiUrl: 'https://api.offshore.l3s.de'
};
```

Listing 11.11: Production environment configuration

11.6.2 Runtime Configuration

For dynamic configuration without rebuilding, environment variables can be injected at container runtime:

```
docker run -d -p 80:80 \
  -e API_URL=https://new-api.example.com \
  l3s-offshore-frontend:latest
```

Listing 11.12: Runtime environment injection

11.7 Troubleshooting

11.7.1 Common Issues

Blank page after deployment: Check nginx configuration and ensure `try_files` fallback is configured for SPA routing.

API connection refused: Verify the backend URL in environment configuration and check CORS settings on the backend.

Assets not loading: Ensure `base href` in `index.html` matches the deployment path.

Container exits immediately: Check logs with `docker logs <container-id>` and verify nginx configuration syntax.

11.7.2 Logging

Access container logs for debugging:

```
# Follow logs in real-time
docker logs -f offshore-frontend

# Last 100 lines
docker logs --tail 100 offshore-frontend
```

Listing 11.13: Viewing container logs

Chapter 12

Related Work

This chapter discusses existing tools and approaches for Petri net visualization and simulation, and positions the L3S-Offshore-2 Frontend within this landscape.

12.1 Desktop Petri Net Tools

12.1.1 WoPeD (Workflow Petri Net Designer)

WoPeD is a widely-used open-source tool for editing and analyzing Workflow Petri nets. It provides:

- Graphical editor for place/transition nets
- Structural analysis (e.g., place invariants)
- Token game simulation
- PNML import/export

Comparison: Unlike WoPeD, the L3S-Offshore-2 Frontend runs entirely in the browser, enabling collaboration without software installation. However, WoPeD offers more advanced analysis algorithms.

12.1.2 CPN Tools

CPN Tools is a tool for editing, simulating, and analyzing Colored Petri Nets (CPNs). It provides hierarchical modeling and state space analysis.

Comparison: CPN Tools supports more expressive Petri net types but requires desktop installation. The L3S frontend focuses on simpler P/T nets with web-based accessibility.

12.2 Web-Based Petri Net Tools

12.2.1 i-love-petrinets (Original Fork)

The L3S-Offshore-2 Frontend is based on a fork of the `i-love-petrinets` project, an Angular-based PNML viewer. The original provided:

- PNML parsing and basic visualization
- Drag-and-drop editing
- Place invariant analysis

Extensions in this project:

- Backend integration for simulation
- Animated firing sequence playback
- Offshore-specific parameter interface
- Production-ready deployment with Docker
- Significant UX improvements (zoom, pan, tab state management)

12.2.2 Cytoscape.js

Cytoscape.js is a general-purpose graph visualization library that has been extended for Petri nets. However, it lacks native PNML support and Petri net semantics.

12.3 Simulation Backends

12.3.1 gspn4py

The L3S backend uses `gspn4py`, a Python library for Generalized Stochastic Petri Nets. It provides:

- Stochastic simulation with exponential delays
- Firing sequence generation
- Performance analysis

12.3.2 pm4py

pm4py is a Python library for process mining that also supports Petri net operations. It was considered during the project's initial phase but the focus shifted to custom simulation for offshore scheduling.

12.4 Summary

The L3S-Offshore-2 Frontend fills a niche for web-based Petri net visualization with domain-specific features for offshore logistics research. While desktop tools offer more advanced analysis, the web-based approach prioritizes accessibility and collaboration.

Chapter 13

Conclusion and Future Work

This chapter summarizes the achievements of the L3S-Offshore-2 Frontend project and outlines directions for future development.

13.1 Summary

This documentation has described the development of a web-based frontend for Petri net visualization and simulation, created as part of the L3S-Offshore-2 research project.

13.1.1 Achievements

The following goals were accomplished:

1. **Functional Visualization:** The application successfully renders Petri nets from PNML files with interactive editing capabilities.
2. **Simulation Integration:** Full integration with the Flask backend enables multi-run simulations with firing sequence animation.
3. **Offshore Parameter Interface:** A dedicated tab allows researchers to configure domain-specific parameters for scheduling models.
4. **Production Deployment:** The application is containerized and deployed on L3S infrastructure, accessible via web browser.
5. **Code Quality:** Extensive JSDoc documentation and English comments ensure maintainability for future developers.

13.1.2 Technical Highlights

The most technically challenging aspects that were solved:

- **Viewport-centered zoom:** Implementing intuitive zoom behavior with ResizeObserver integration
- **Tab-state management:** Ensuring UI consistency when switching between different workflows
- **Token reset logic:** Preserving initial markings across simulation runs
- **Reactive forms integration:** Solving performance issues with Angular's form system

13.2 Limitations

The current implementation has the following limitations:

- **No data persistence:** All data is stored in-memory and lost on page reload
- **Limited to P/T nets:** Higher-level Petri net types (colored, hierarchical) are not supported
- **Single-user:** No collaborative editing or user accounts
- **Visual overlaps:** Large nets may have overlapping elements with the current layout

13.3 Future Work

The following enhancements could be pursued in future development:

13.3.1 Short-Term (High Priority)

- **Interactive Miner Integration:** Connect to pm4py's interactive miner for process discovery
- **Analysis Endpoints:** Display conformance checking results from the backend
- **Export Functions:** Enable export of simulation results as CSV/JSON

13.3.2 Medium-Term (Nice-to-Have)

- **Undo/Redo:** Implement history management for the Build tab
- **Data Persistence:** Add optional browser-based storage (IndexedDB)
- **Visual Improvements:** Reduce element overlaps with improved layout algorithms

13.3.3 Long-Term (Research Extensions)

- **Colored Petri Nets:** Extend the data model and visualization for CPNs
- **Real-Time Monitoring:** WebSocket-based live updates during long-running simulations
- **Multi-User Collaboration:** Shared editing with conflict resolution

13.4 Acknowledgements

This project was developed at the L3S Research Center, Leibniz Universität Hannover, under the supervision of Peng Qian. The author thanks the original developers of the i-love-petrinets project for providing the initial codebase.

Appendix A

Appendix: API Reference

This appendix provides a reference for the backend API endpoints used by the frontend.

A.1 Simulation Endpoints

A.1.1 Simple Simulation

Endpoint: `POST /simulation-petri-nets/simple-sim`

Description: Executes a stochastic simulation of a Petri net

Request Body: PNML content as string or file upload

Response: JSON containing firing sequence and optional detailed log

```
{
  "firing_seq": ["t1", "t3", "t2", "t1", "t4"],
  "detailed_log": [
    {"step": 0, "marking": {"p1": 1, "p2": 0}},
    {"step": 1, "marking": {"p1": 0, "p2": 1}}
  ]
}
```

Listing A.1: Example response from simple-sim

A.1.2 Example Models

Endpoint: `GET /simulation-petri-nets/example-models`

Description: Retrieves a list of available example PNML files

Response: Array of model names

Endpoint: `GET /simulation-petri-nets/example-models/<name>`

Description: Retrieves the PNML content of a specific example model

Response: PNML XML as string

A.2 Planning Endpoints

A.2.1 Default Parameters

Endpoint: `GET /dto/planning`

Description: Retrieves default parameter values for offshore scheduling

Response: JSON object with parameter definitions and defaults

A.2.2 Schedule Calculation

Endpoint: `POST /offshore-plan/calculate-schedule`

Description: Calculates an optimal schedule based on provided parameters

Request Body: JSON with scenario and workforce configuration

Response: Schedule with operation timings

A.3 Configuration Files

A.3.1 Angular Environment

`src/environments/environment.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:9040'
};
```

Listing A.2: Development environment configuration

A.3.2 Docker Compose

`docker-compose.yml` for local development with backend:

```
version: '3.8'
services:
  frontend:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - backend
  backend:
```

```
10      image: l3s-offshore-backend:latest
11      ports:
12        - "9040:9040"
```

Listing A.3: Docker Compose configuration

Bibliography

[1] Google. Angular documentation. https://angular.io/docs, 2024. Accessed: 2025-11-01.

[2] Google. Angular material. https://material.angular.io/, 2024. Accessed: 2025-11-01.

[3] M. A. Marsan, G. Conte, and G. Balbo. A class of generalized stochastic petri nets for the performance evaluation of multiprocessor systems. *ACM Transactions on Computer Systems*, 2(2):93–122, 1984.

[4] T. Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.

[5] C. A. Petri. Kommunikation mit automaten. *Dissertation, Universität Bonn*, 1962. English translation: Communication with Automata, Technical Report RADC-TR-65-377, Griffiss Air Force Base, New York, 1966.

[6] ReactiveX. Rxjs - reactive extensions library for javascript. https://rxjs.dev/, 2024. Accessed: 2025-11-01.

[7] K. Sugiyama, S. Tagawa, and M. Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.

[8] M. Weber and E. Kindler. The petri net markup language. Technical report, Petri Net Newsletter, 2004. http://www.pnml.org/.